

数字信号处理

(Digital Signal Processing-DSP)

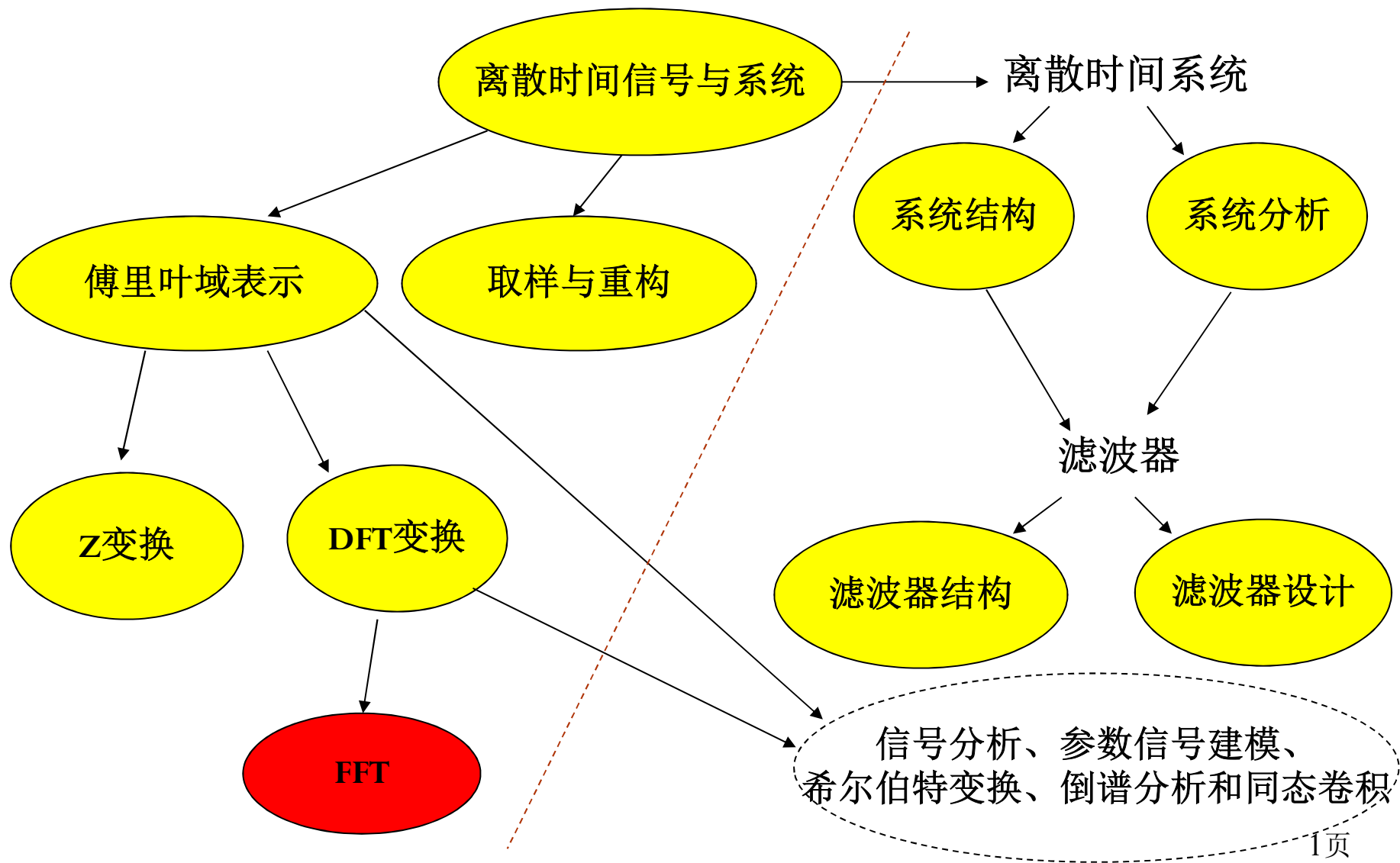
第九章 离散傅里叶变换的计算 (FFT)

谢 翔

邮箱: xiexiang@tsinghua.edu.cn

电话: 62784683

第9章 离散傅里叶变换计算 (FFT)



引言

1、DFT计算量太大(与N的平方成正比), 很难实时地处理问题, 因此, 引出了如何提高计算效率?

2、DFT算法的评价: 计算复杂度 (乘法和加法次数)、资源

3、DFT的有效算法: 快速傅里叶变换
(Fast Fourier Transform, FFT)

4、FFT的意义:

1) DFT理论得以在实际中广泛应用

2) 数字信号处理技术普及的重要因素之一

注: FFT不是一种新的变换, 仅是一种快速算法!

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

9.1、离散傅里叶变换的直接计算

对一个 N 点序列进行DFT有：

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, n = 0, 1, 2, \dots, N-1$$

对应的IDFT为：

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k)W_N^{-kn}, k = 0, 1, 2, \dots, N-1$$

DFT正反变换的形式相同（仅符号和系数不同），计算量因此相同

DFT计算量分析：

- 1) 每一个 $X(k)$ 的计算量：需 N 次复数乘法和 $N-1$ 次复数加法
- 2) 所有 $X(k)$ 的计算量：需 N^2 次复数乘法和 $N(N-1)$ 次复数加法
- 3) 总的实数计算量：需 $4N^2$ 次乘法和 $2N(N-1) + 2N^2 = 4N^2 - 2N$ 次加法

DFT计算量与 N^2 成正比， N 很大时，计算次数非常大，难于实时实现。

现有提高DFT计算效率的方法，均需要利用 W_N^{kn} 的特性：比如对称性、周期性可约性等

$$W_N^{nk} = e^{-j\frac{2\pi}{N}nk}$$

1)对称性: $(W_N^{nk})^* = W_N^{-nk} = W_N^{(N-n)k}$

2)周期性: $W_N^{nk} = W_N^{(N+n)k} = W_N^{n(k+N)}$

3)可约性: $W_N^{nk} = W_{mN}^{mnk}, W_N^{nk} = W_{N/m}^{nk/m}$

4)特殊点: $W_N^0 = 1, W_N^{\frac{N}{2}} = -1, W_N^{(k+N/2)} = -W_N^k$

利用对称性特性，可使得某些计算项合并或无需乘法和加法，但降低计算量的效果不明显。

利用周期性特性，可使得计算量显著降低。

Cooley和Tukey于1965年发表了一个快速的算法，由Cooley提出思想，Tukey完成实际的算法（程序）

----- 在信号处理应用领域具有划时代的意义

FFT算法对于 N 点DFT, 仅需
 $(N/2)\log_2 N$ 次复数乘法运算
和 $N\log_2 N$ 次复数加法。

An Algorithm for the Machine Calculation of Complex Fourier Series

By James W. Cooley and John W. Tukey

An efficient method for the calculation of the interactions of a 2^m factorial experiment was introduced by Yates and is widely known by his name. The generalization to 3^m was given by Box et al. [1]. Good [2] generalized these methods and gave elegant algorithms for which one class of applications is the calculation of Fourier series. In their full generality, Good's methods are applicable to certain problems in which one must multiply an N -vector by an $N \times N$ matrix which can be factored into m sparse matrices, where m is proportional to $\log N$. This results in a procedure requiring a number of operations proportional to $N \log N$ rather than N^2 . These methods are applied here to the calculation of complex Fourier series. They are useful in situations where the number of data points is, or can be chosen to be, a highly composite number. The algorithm is here derived and presented in a rather different form. Attention is given to the choice of N . It is also shown how special advantage can be obtained in the use of a binary computer with $N = 2^m$ and how the entire calculation can be performed within the array of N data storage locations used for the given Fourier coefficients.

Consider the problem of calculating the complex Fourier series

$$(1) \quad X(j) = \sum_{k=0}^{N-1} A(k) \cdot W^{jk}, \quad j = 0, 1, \dots, N-1,$$

where the given Fourier coefficients $A(k)$ are complex and W is the principal N th root of unity,

$$(2) \quad W = e^{2\pi i/N}.$$

A straightforward calculation using (1) would require N^2 operations where "operation" means, as it will throughout this note, a complex multiplication followed by a complex addition.

The algorithm described here iterates on the array of given complex Fourier amplitudes and yields the result in less than $2N \log_2 N$ operations without requiring more data storage than is required for the given array A . To derive the algorithm, suppose N is a composite, i.e., $N = r_1 \cdot r_2$. Then let the indices in (1) be expressed

$$(3) \quad \begin{aligned} j &= j_1 r_1 + j_0, & j_0 &= 0, 1, \dots, r_1 - 1, & j_1 &= 0, 1, \dots, r_2 - 1, \\ k &= k_1 r_2 + k_0, & k_0 &= 0, 1, \dots, r_2 - 1, & k_1 &= 0, 1, \dots, r_1 - 1. \end{aligned}$$

Then, one can write

$$(4) \quad X(j_1, j_0) = \sum_{k_0} \sum_{k_1} A(k_1, k_0) \cdot W^{j_1 r_1 k_1 + j_0 k_0}.$$

Received August 17, 1964. Research in part at Princeton University under the sponsorship of the Army Research Office (Durham). The authors wish to thank Richard Garwin for his essential role in communication and encouragement.

297

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

9.2、Goertzel算法

利用 W_N^{kn} 的周期特性，将DFT的计算表示为线性滤波运算

$$X(k) = W_N^{-kN} \sum_{r=0}^{N-1} x(r) W_N^{kr} = \sum_{r=0}^{N-1} x(r) W_N^{-k(N-r)}, r = 0, 1, 2, \dots, N-1$$

再令：

$$y_k(n) = \sum_{r=0}^{N-1} x(r) W_N^{-k(n-r)} u(n-r) = x(n) * h_k(n), \text{其中 } h_k(n) = W_N^{-kn} u(n)$$

当 $n = N$ 时，该滤波器的输出为DFT在频率 $\omega_k = 2\pi k/N$ 处的值

$$X(k) = y_k(n) \big|_{n=N}, \quad y_k(0) = x(0)$$

滤波器的系统函数可表示为： $H_k(z) = \frac{1}{1 - W_N^{-k} z^{-1}}$ 4N次实数乘法和加法，仅存储1个复数系数！

该滤波器在单位圆上的频率 $\omega_k = 2\pi k/N$ 处有一个极点，因此，将输入数据输入到 N 个单极点并行滤波器（谐振器）组，可计算整个 N 点DFT，其中，每一滤波器在相应的DFT频率上有一极点

利用滤波器的差分方程进行递推计算：

$$y_k(n) = W_N^k y_k(n-1) + x(n), y_k(-1) = 0$$

$$X(k) = y_k(N), k = 0, 1, 2, \dots, N-1$$

能否进一步降低计算量呢？

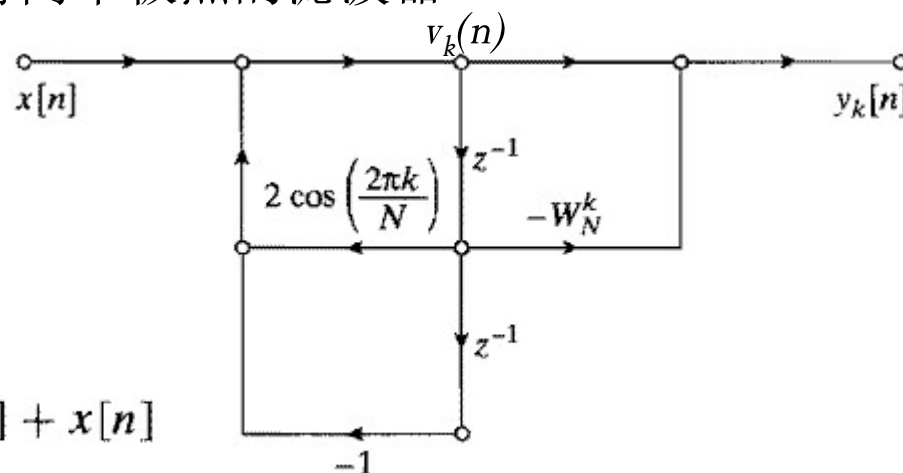
将复共轭极点的谐振器结合成对，具有两个极点的滤波器

$$\begin{aligned} H_k(z) &= \frac{1 - W_N^k z^{-1}}{(1 - W_N^{-k} z^{-1})(1 - W_N^k z^{-1})} \\ &= \frac{1 - W_N^k z^{-1}}{1 - 2 \cos(2\pi k/N) z^{-1} + z^{-2}} \end{aligned}$$

$$v_k[n] = 2 \cos(2\pi k/N) v_k[n-1] - v_k[n-2] + x[n]$$

$$X[k] = y_k[n] \Big|_{n=N} = v_k[N] - W_N^k v_k[N-1]$$

初始条件： $v_k(-1)=0, v_k(-2)=0$



优点： 1) 输入复数序列 $x(n)$ ，仅 $2N + 4$ 次实数乘法可得到 $X(k)$ ，约直接法的一半，而 $x(n)$ 为实序列时，仅 $N + 2$ 次乘法，系数只需存储 3 个，可得到 $X(k)$ ，因此求 DFT 的 N 个点时，总乘法次数 N^2 关系。2) 若只需计算 DFT 的 M 个点，且 $M < 1/2 \log_2 N$ ，Goertzel 算法 (MN 次乘法) 的优势明显，否则，FFT 算法 $1/2 N \log_2 N$ 有优势。

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel 算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
 - 1、时间抽取概念
 - 2、8点基2FFT的流图结构
 - 3、同址计算结构
 - 4、其它结构
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

9.3、按时间抽取(Decimation in Time)的FFT算法

FFT的核心思想：

把计算量与 N^2 成正比的DFT运算，利用复指数 W_N^{kn} 的特性，分解成较短的子序列进行计算（达到降低计算量至 $N\log_2 N$ 效果），组合后得到最终结果。也即：

- 1) 把一个序列分为长度减半的偶序列和奇序列，原序列的DFT就由这两个 $N/2$ 序列求得；
- 2) 进一步把 $N/2$ 序列分解成两个 $N/4$ 序列，一直分解到单点序列。

因分解方法的不同产生了多种DFT的快速算法！

假设序列点数 $N = 2^v$ ， v 为整数，也称**基-2FFT**算法，由 N 为偶整数，将序列分解为奇数点和偶数点两组序列DFT可以表示为：

$$X[k] = \sum_{n=0}^{N-1} x[n]W_N^{nk} = \sum_{n \text{ 为偶数}} x[n]W_N^{nk} + \sum_{n \text{ 为奇数}} x[n]W_N^{nk}$$

作变量代换, $n=2r$ (偶数); $n=2r+1$ (奇数)

$$\begin{aligned} X[k] &= \sum_{r=0}^{(N/2)-1} x[2r]W_N^{2rk} + \sum_{r=0}^{(N/2)-1} x[2r+1]W_N^{(2r+1)k} \\ &= \sum_{r=0}^{(N/2)-1} x[2r](W_N^2)^{rk} + W_N^k \sum_{r=0}^{(N/2)-1} x[2r+1](W_N^2)^{rk} \end{aligned}$$

因为,

$$W_N^2 = e^{-2j(2\pi/N)} = e^{-j2\pi/(N/2)} = W_{N/2}$$

上式可写为:

$$\begin{aligned} X[k] &= \sum_{r=0}^{(N/2)-1} x[2r]W_{N/2}^{rk} + W_N^k \sum_{r=0}^{(N/2)-1} x[2r+1]W_{N/2}^{rk} \\ &= G[k] + W_N^k H[k] \end{aligned} \quad k = 0, 1, \dots, N-1$$

因此N点DFT可表示为:

$$X(k) = G(k) + W_N^k H(k), k = 0, 1, \dots, N-1$$

其中:

$G[k]$ —— 原序列偶数点的 $N/2$ 点DFT (周期为 $N/2$)

$H[k]$ —— 原序列奇数点的 $N/2$ 点DFT (周期为 $N/2$)

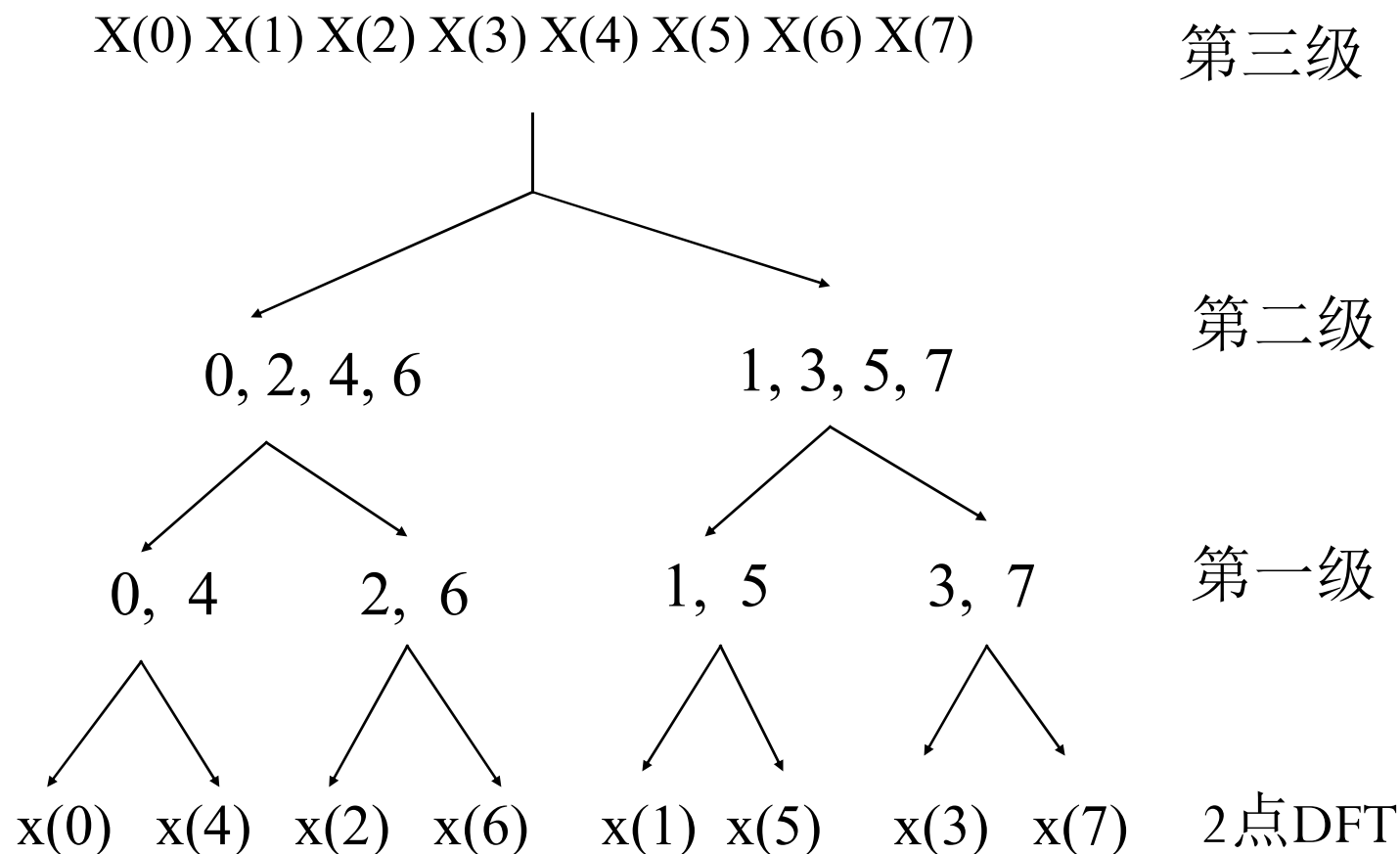
利用 $G(k+N/2)=G(k)$, $H(k+N/2)=H(k)$,

当 $k \geq N/2-1$ 时, $X(k)$ 中的 $G(k)$ 与 $H(k)$ 不用再计算, 因此可降低近一半的计算量

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
 - 1、时间抽取概念
 - 2、8点基2FFT的蝶形结构
 - 3、同址计算结构
 - 4、其它结构
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

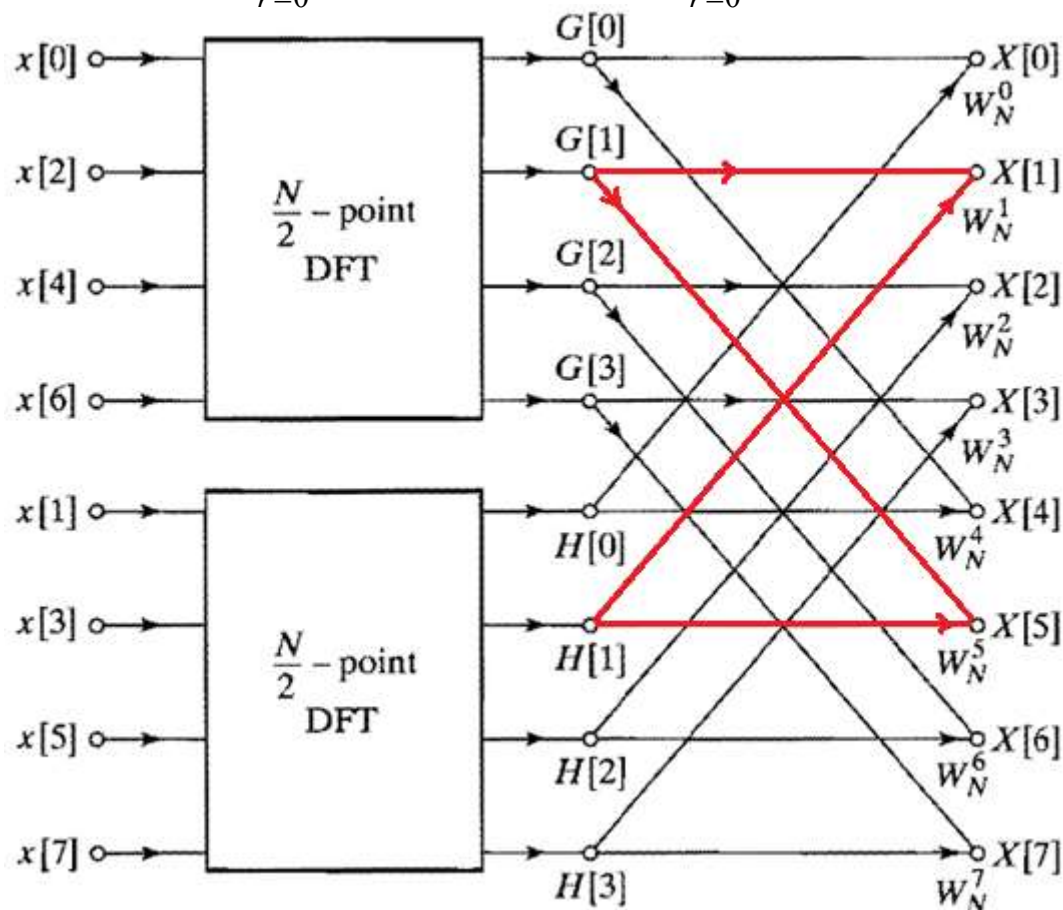
N=8的基2FFT



输出序列的序号是正常排序，输入是非正常

例子：当 $N=8$ 时，基2FFT算法的计算过程可用下图表示：

$$X(k) = \sum_{r=0}^{N/2-1} x(2r)W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x(2r+1)W_{N/2}^{rk}$$



$$X(k) = G(k) + W_N^k H(k),$$

$$X(k + N/2) = G(k) + W_N^{k+N/2} H(k)$$

其中： $k = 0, 1, \dots, N/2 - 1$

直观理解：只要计算一半的点

计算量分析：

两个 $N/2$ 点DFT+组合

复数乘法次数：

$$2(N/2)^2 + N = N^2/2 + N$$

复数加法次数：

$$2(N/2 - 1)(N/2) + N = N^2/2 + N$$

组合产生的计算量：

复乘 N 次，复加 N 次

与直接法相比较，当 N 比较大时，运算量降低接近一半。

组合部分： $N/2$ 个蝶形结构

核心是利用了 W_N 的周期性

在前面基础上，可以进一步作上述的分解，即每一个 $N/2$ 点DFT可以分解为两个（按奇、偶） $N/4$ 点DFT，再组合。

可表示为：
$$G[k] = \sum_{r=0}^{(N/2)-1} g[r] W_{N/2}^{rk} = \sum_{l=0}^{(N/4)-1} g[2l] W_{N/2}^{2lk} + \sum_{l=0}^{(N/4)-1} g[2l+1] W_{N/2}^{(2l+1)k}$$

即：
$$G[k] = \sum_{l=0}^{(N/4)-1} g[2l] W_{N/4}^{lk} + W_{N/2}^k \sum_{l=0}^{(N/4)-1} g[2l+1] W_{N/4}^{lk}$$

$$H[k] = \sum_{l=0}^{(N/4)-1} h[2l] W_{N/4}^{lk} + W_{N/2}^k \sum_{l=0}^{(N/4)-1} h[2l+1] W_{N/4}^{lk}$$

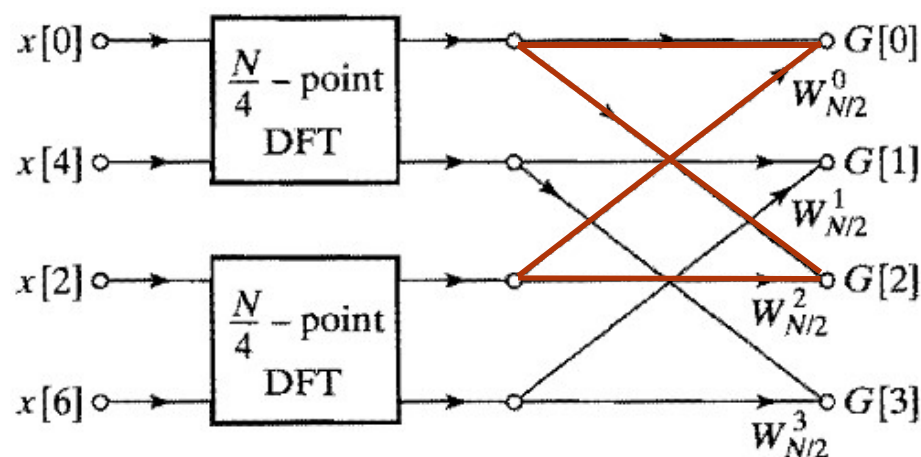
$g[2l]$ 和 $g[2l+1]$ 的 $N/4$ 点DFT+组合运算= $N/2$ 点DFT—— $G[k] = \frac{N^2}{8} + \frac{N}{2}$

$h[2l]$ 和 $h[2l+1]$ 的 $N/4$ 点DFT+组合运算= $N/2$ 点DFT—— $H[k] = \frac{N^2}{8} + \frac{N}{2}$

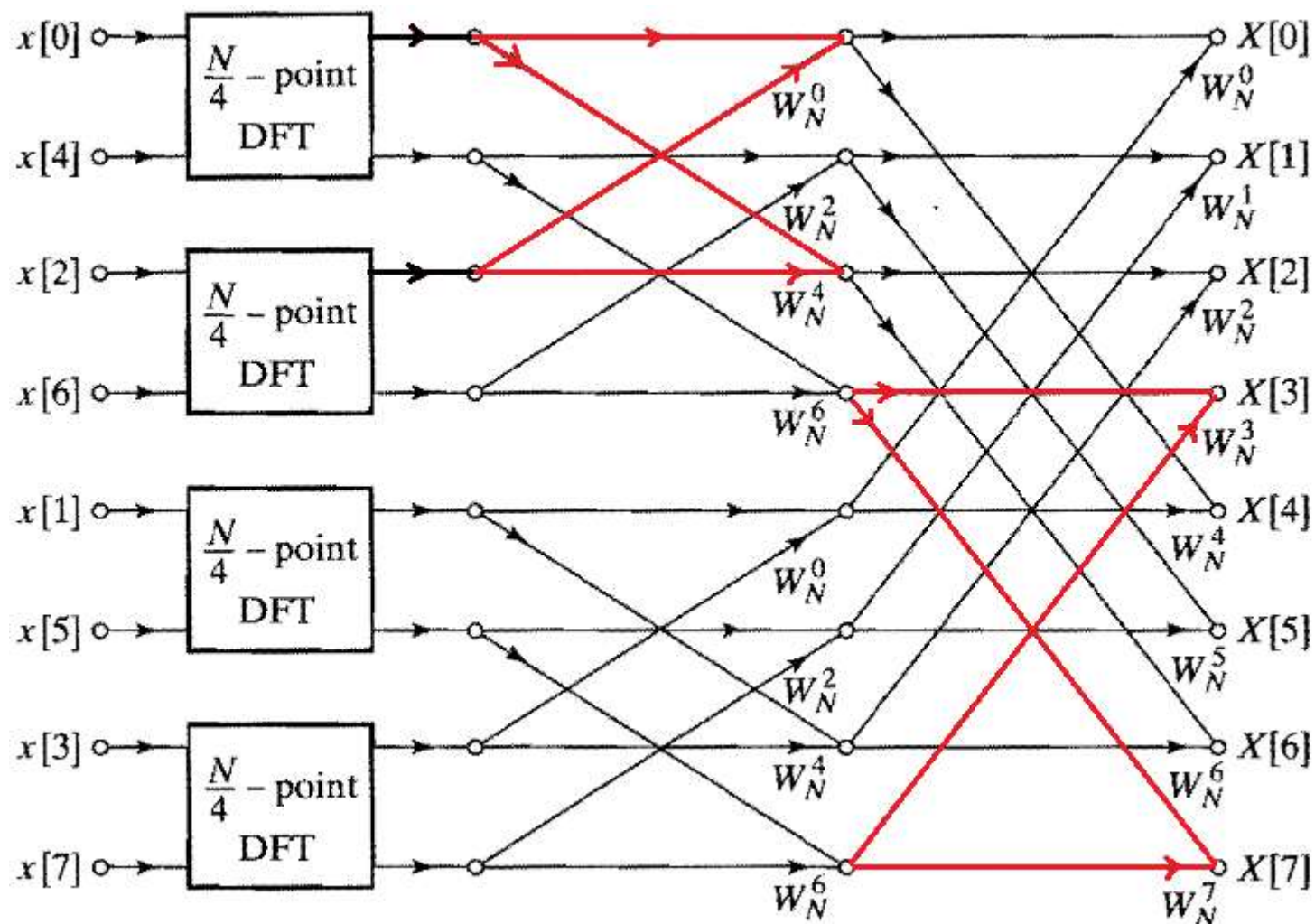
组合产生的计算量：
复乘 N 次，复加 N 次

$G[k]$ 计算见右结构图：

组合部分： $4/2=2$ 个蝶形结构



8点DFT计算过程可表示为下图形式：(利用 $W_{N/2} = W_N^2$)

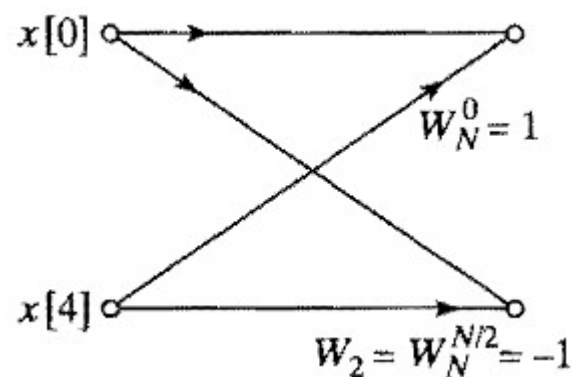


基本单元是蝶形结构，每一级蝶形个数都为 $N/2$

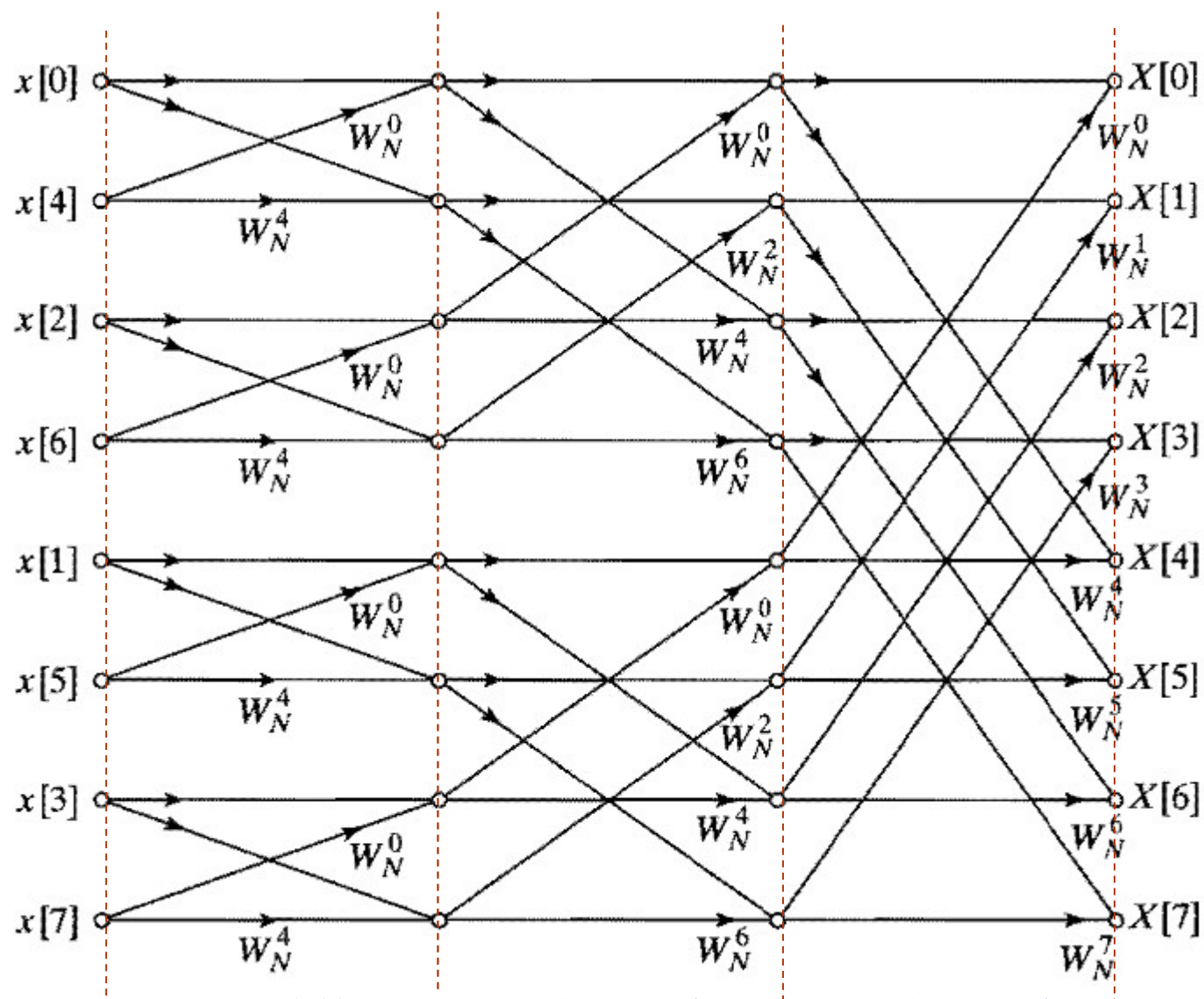
对于 $N = 8$ 来说, $N/4 = 2$, 2点的DFT一般形式可表示为:

$$X'(k) = \sum_{n=0}^1 x'(n)W_2^{nk} = x'(0)W_2^0 + x'(1)W_2^k, \quad k = 0,1$$

2点的DFT —— 组合运算, 可用下图表示:



因此，8点的DFT计算过程可表示为如下图：



2点DFT结构
可理解为组合结构

N/4点DFT输
出的组合结构

N/2点DFT输
出的组合结构

若序列点数： $N = 2^v$ ，分解的最终是计算2点的DFT+组合运算

分解级数： $v = \log_2 N$ ，每一级为N次复数乘法+N次复数加法，
因为每一级仅仅存在组合运算

FFT总的运算量约为： $N \log_2 N$ 次复数乘法+ $N \log_2 N$ 次复数加法

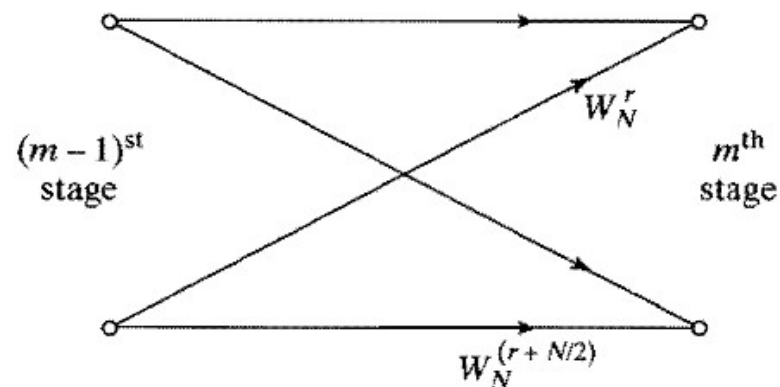
利用 W_N^{kn} 的特性，实际FFT复数乘法次数可以再降低一半

每一级的组合运算可表示为 $N/2$ 个蝶形运算（也即组合运算），如下图所示：

$$X(k) = G(k) + W_N^k H(k),$$

$$X(k + N/2) = G(k) + W_N^{k+N/2} H(k)$$

其中： $k = 0, 1, \dots, N/2 - 1$

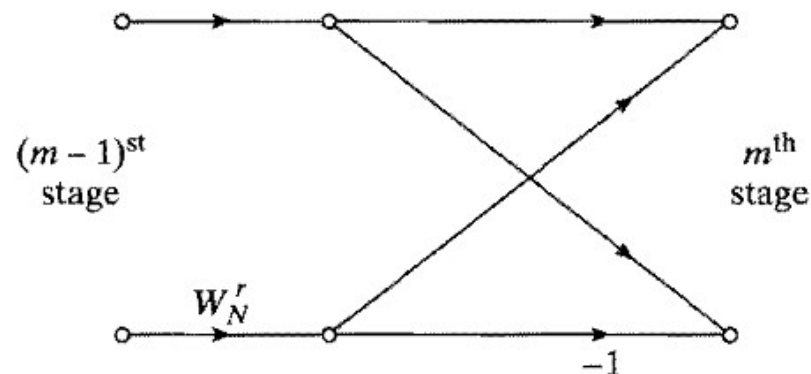


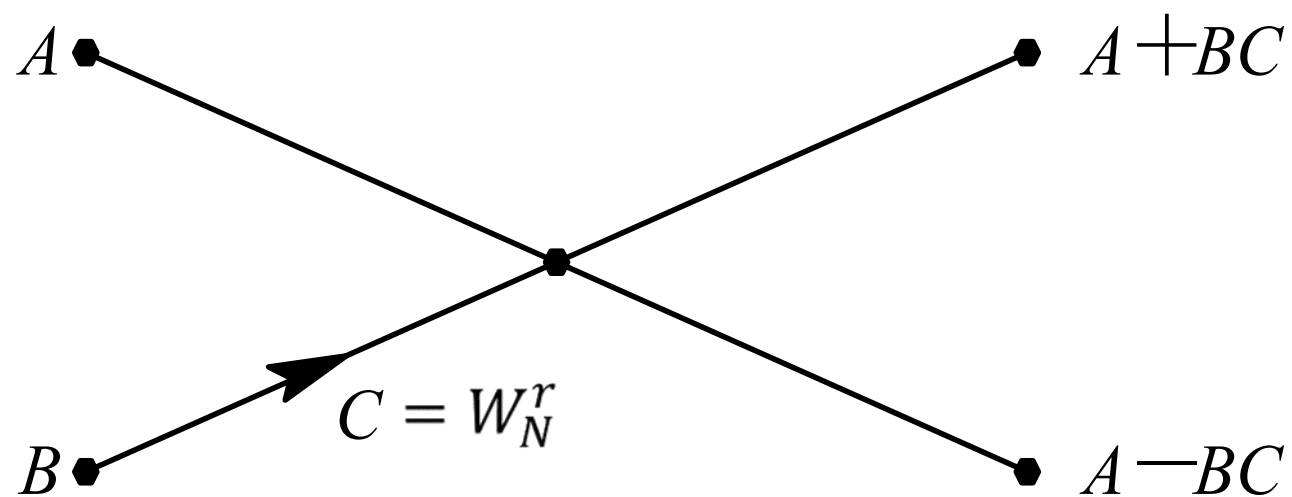
由于： $W_N^{N/2} = e^{-j(2\pi/N)N/2} = e^{-j\pi} = -1$

因此： $W_N^{r+N/2} = W_N^r W_N^{N/2} = -W_N^r$

则该蝶形运算还可表示为：

仅有1次复数乘法
和2次复数加法





图： 另外一种蝶形运算符号

FFT总运算量= $(N/2)\log_2 N$ 次复数乘法+ $N\log_2 N$ 次复数加法

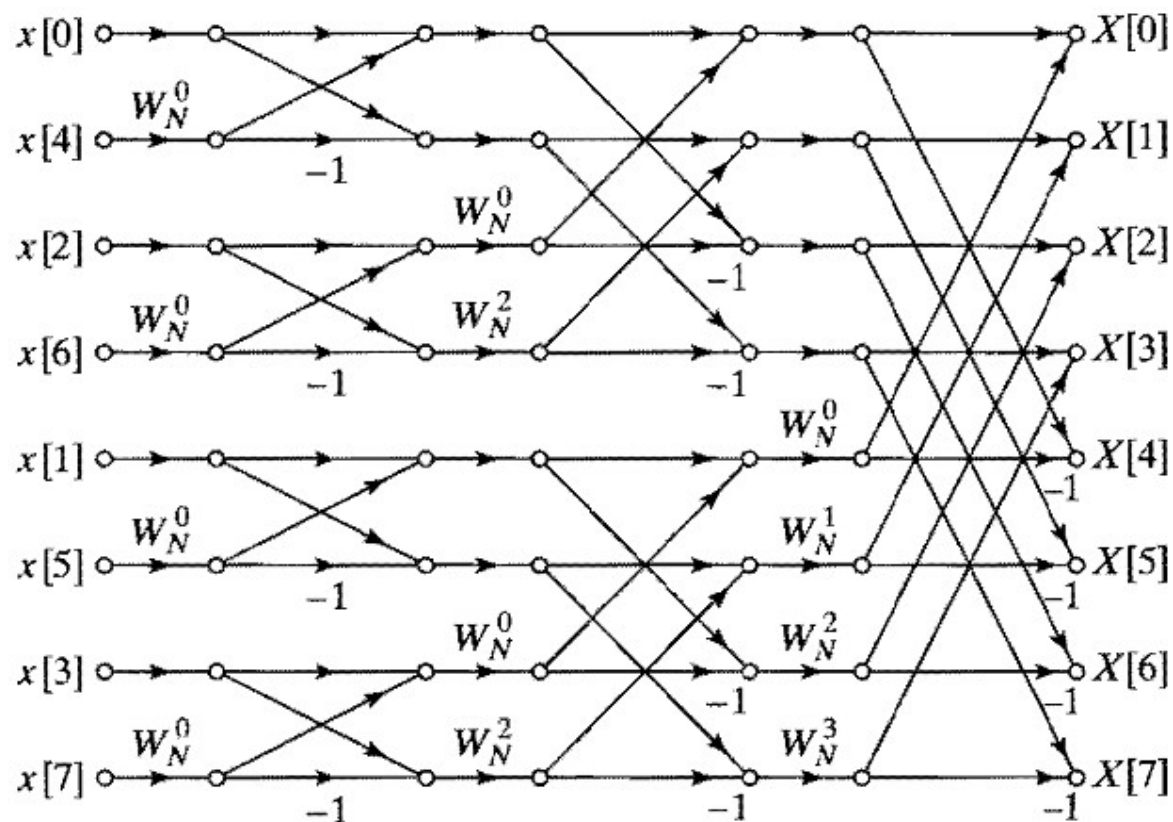
比较 $N = 1024$

直接计算: $N = 2^{10}$, $N^2 = 2^{20} = 1,048,576$ 复数乘法

FFT计算: $(N/2)\log_2 N = 5,120$

减少200多倍的复数乘法, N 越大效果越明显

最终优化后8点的基2FFT算法的流图结构如下所示



同址计算结构

第一级蝶形输入间隔为1，第二级蝶形输入间隔为2，
第三级蝶形输入间隔为4，...

另一种8点基2FFT结构表示方法

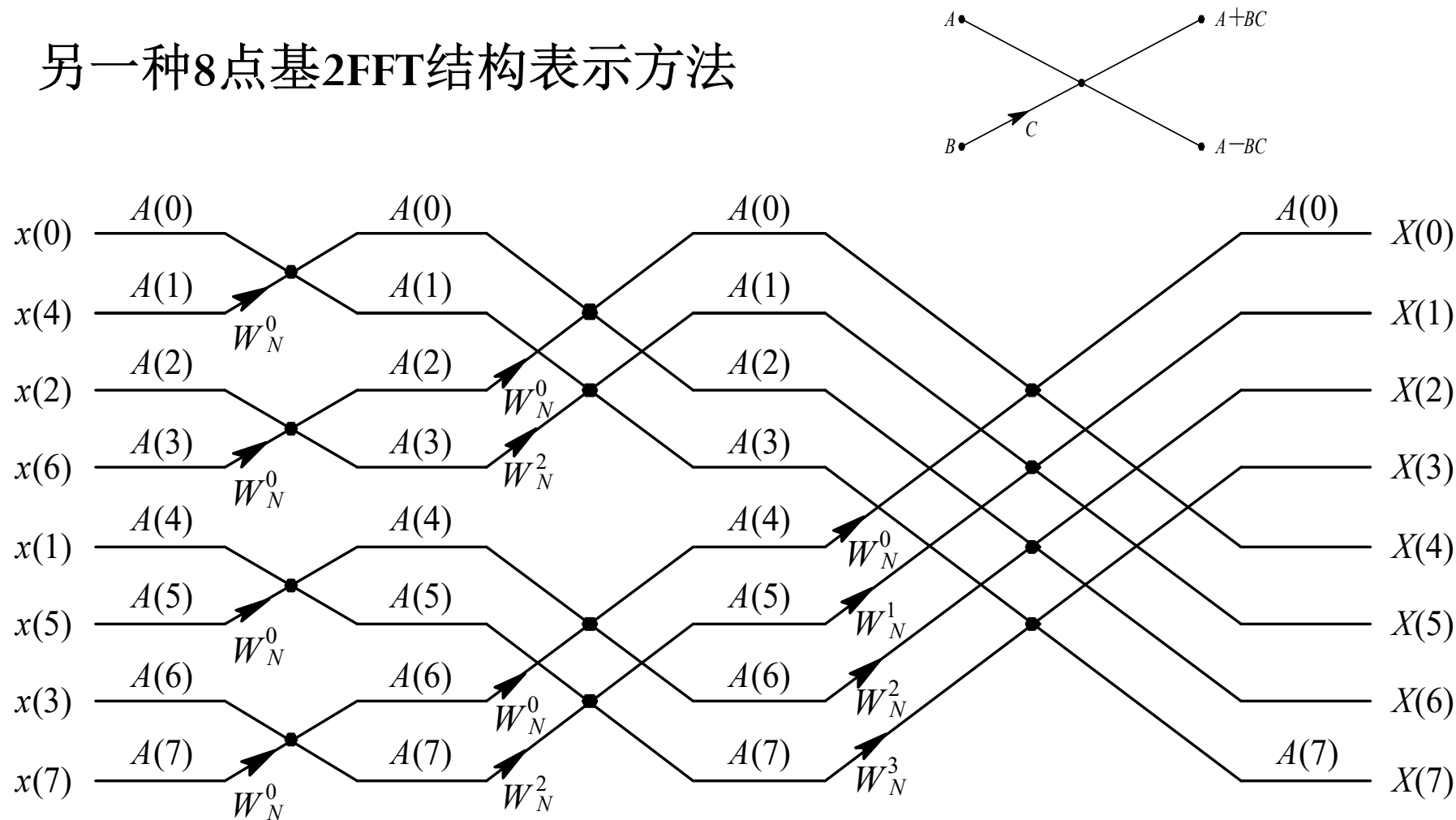


图 N点DIT—FFT运算流图(N=8)

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
 - 1、时间抽取概念
 - 2、8点基2FFT的蝶形结构
 - 3、同址计算结构特点
 - 4、其它结构
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

同址计算结构特点:

基2FFT: 点数 $N = 2^v$, v 级分解运算, 每级 $N/2$ 个蝶形运算

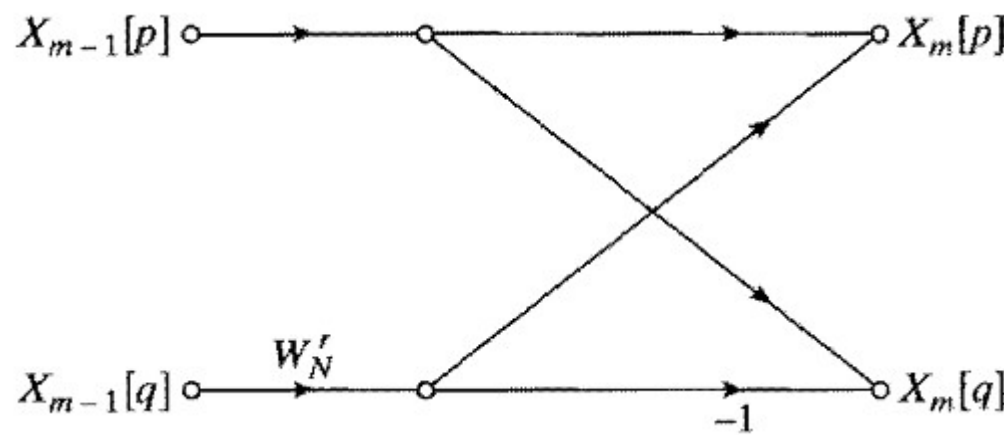
每个蝶形运算: 1次复数乘法, 2次复数加法

整个DFT运算: $(N/2) \log_2 N$ 个蝶形运算

每一级: N 个复数 (输入), $N/2$ 个蝶形运算, N 个复数 (输出)

每个蝶形运算可表示为:

对应方程表示为:


$$\begin{aligned} X_m[p] &= X_{m-1}[p] + W_N^r X_{m-1}[q] \\ X_m[q] &= X_{m-1}[p] - W_N^r X_{m-1}[q] \end{aligned}$$

结论: 蝶形运算输出只与相应输入节点值有关, 而与其他节点无关, 输出值可以覆盖输入值。

因此, 可进行**同址计算**!

m 为节点列数, p 和 q 为节点的行数

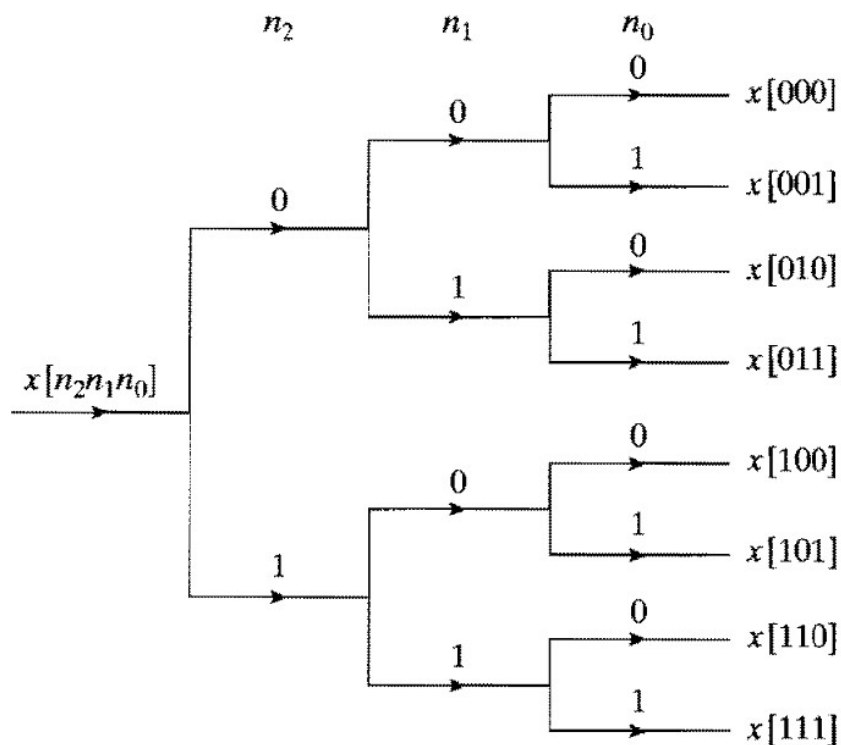
根据前面分析，基2FFT算法的计算过程只需要N个复数寄存器
输入序列放置的位置顺序采用如下所示的倒位序（8点FFT为例）：

$X_0[0] = x[0],$	$X_0[000] = x[000]$
$X_0[1] = x[4],$	$X_0[001] = x[100]$
$X_0[2] = x[2],$	$X_0[010] = x[010]$
$X_0[3] = x[6],$	$X_0[011] = x[110]$
$X_0[4] = x[1],$	$X_0[100] = x[001]$
$X_0[5] = x[5],$	$X_0[101] = x[101]$
$X_0[6] = x[3],$	$X_0[110] = x[011]$
$X_0[7] = x[7].$	$X_0[111] = x[111]$

以 $N = 8$ 为例，将 N 表示为二进制码： $n_2n_1n_0$

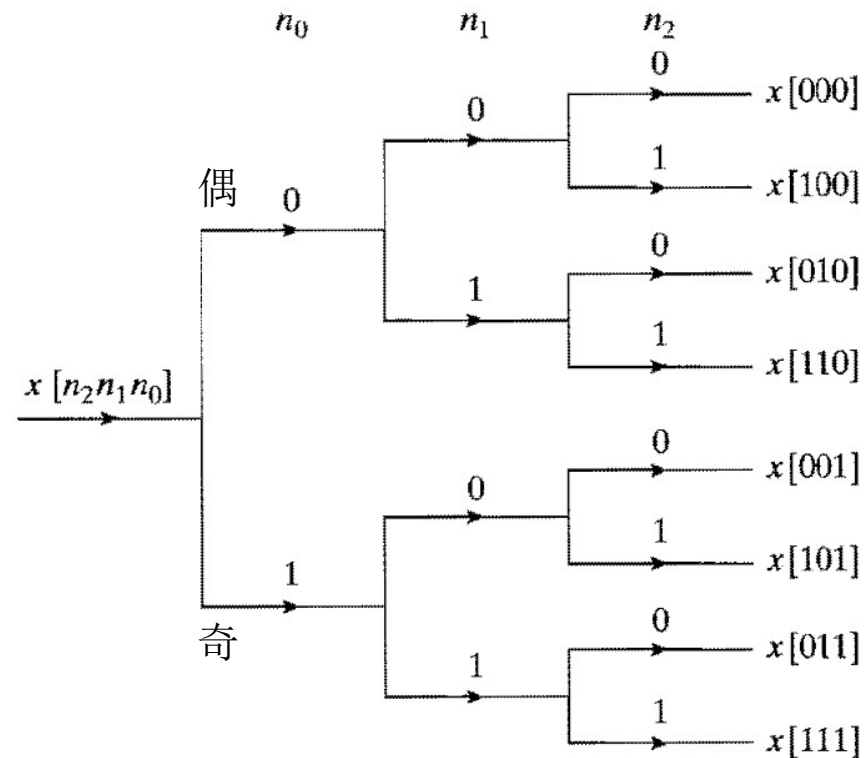
序列 $x[n] = x[n_2n_1n_0]$,

正常排序方法（从高位到低位）



正常排序的树状图

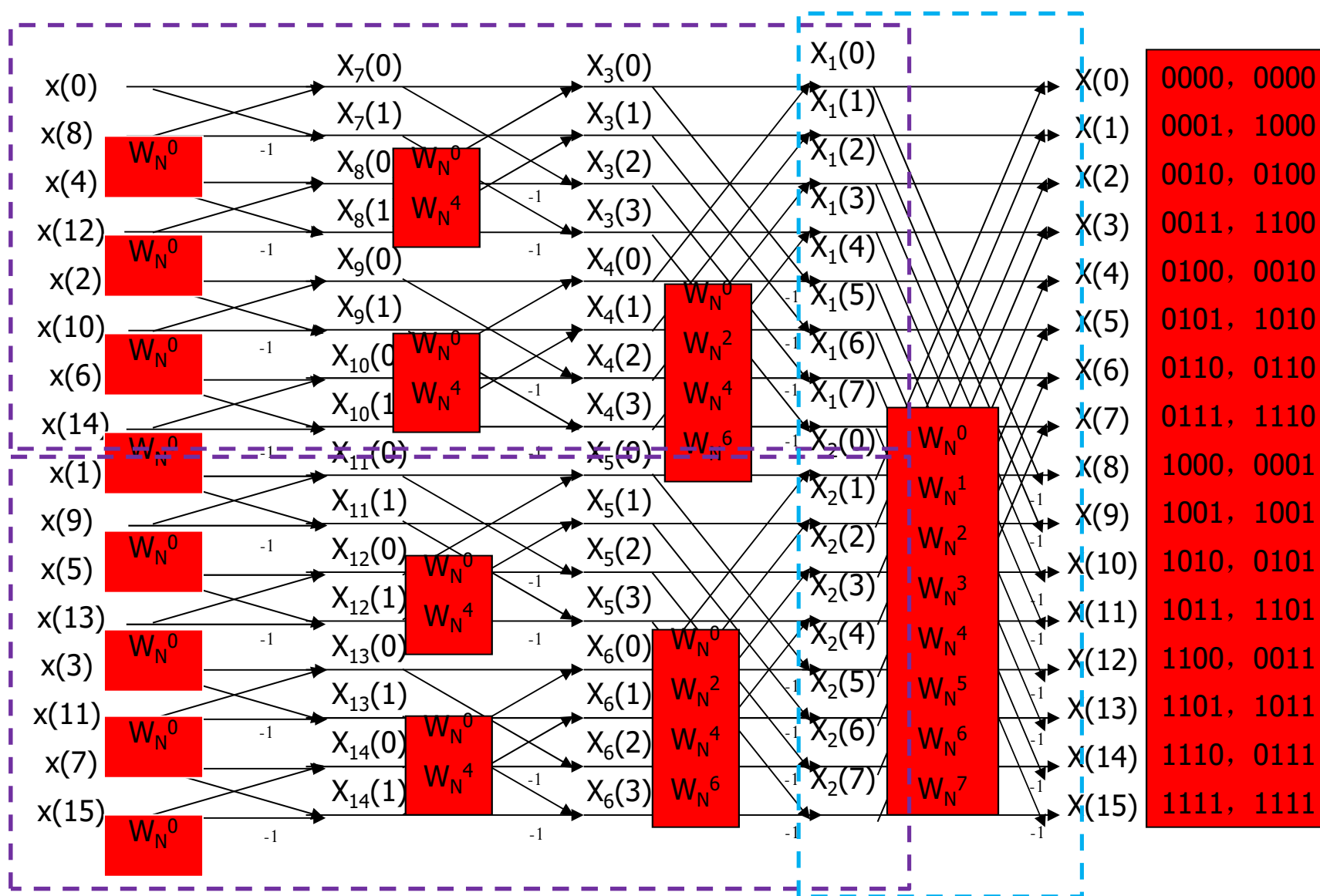
倒序方法（从低位到高位）



倒位排序的树状图

同址计算结构中，输入是倒位序排列，输出是顺序排列，
存储开销只需 N 个复数存储单元！

16点的FFT算法的流图结构



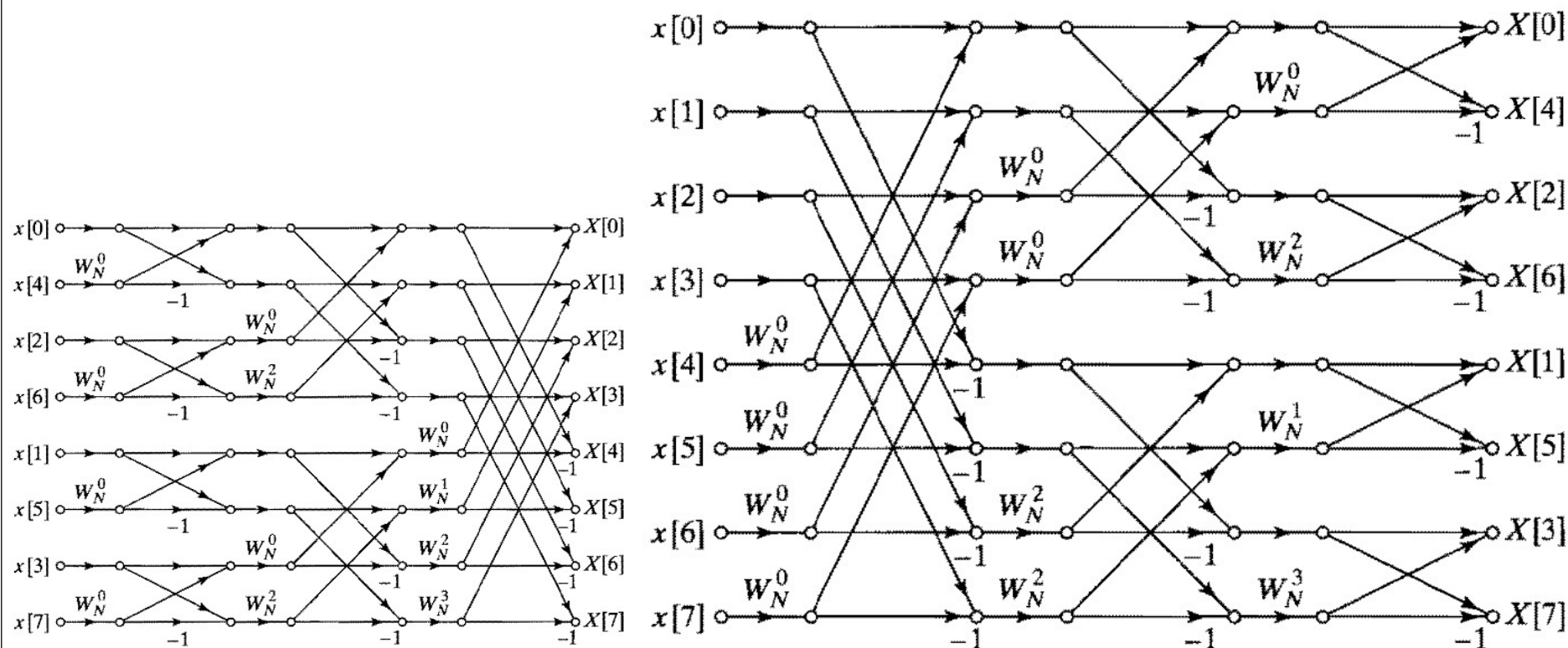
16点基2FFT的同址计算结构

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
 - 1、时间抽取概念
 - 2、8点基2FFT的蝶形结构
 - 3、同址计算结构特点
 - 4、其它结构
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

其它方法的结构:

1) 输入序列正常排序, DFT输出序列是倒位序结构

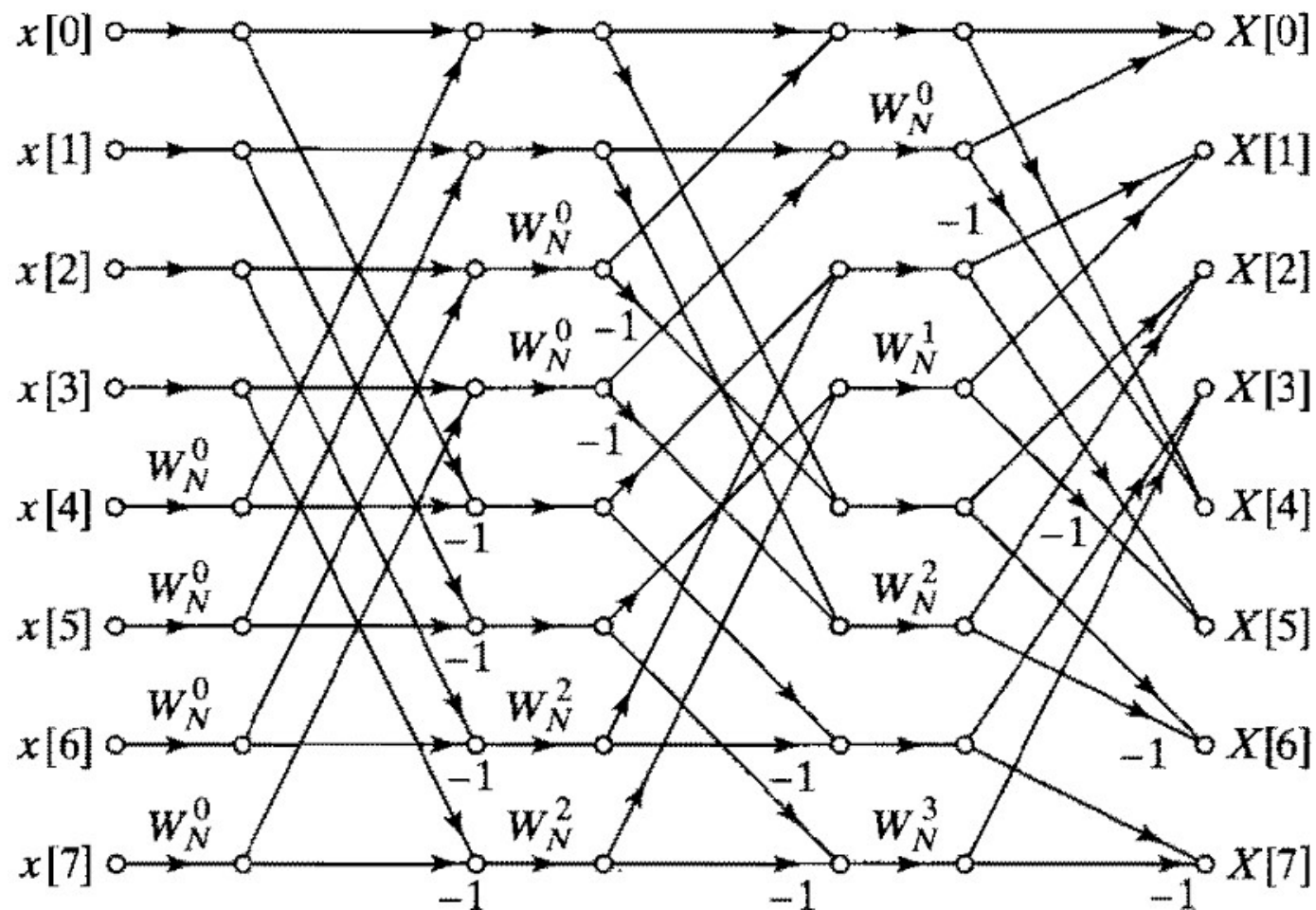


同址计算结构



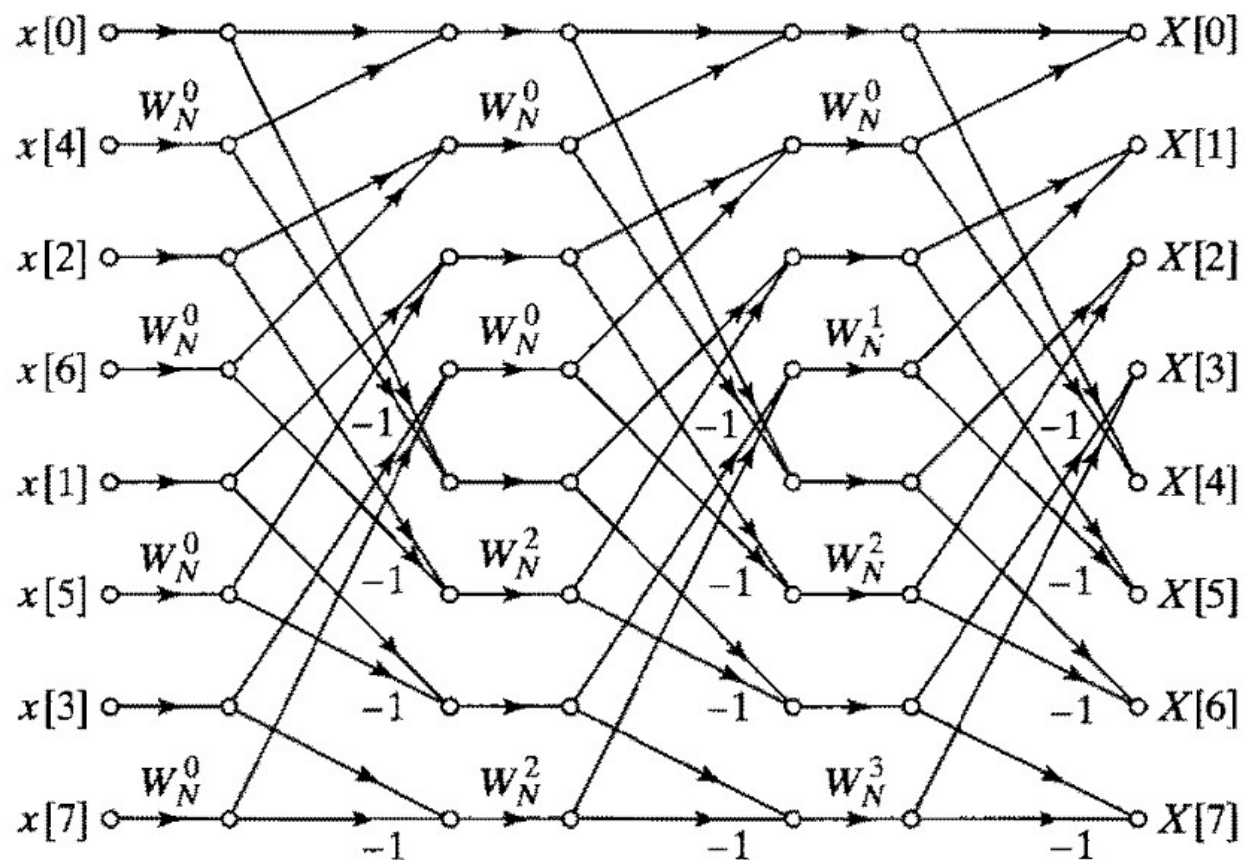
输入顺序, 输出倒位序结构: 各列的蝶形输入的间隔为4, 2, 1

2) 输入与输出序列均正常排序结构



无法进行同址计算，需要两列长度为 N 的复数寄存器
因此从计算角度看，该结构无意义

3) 输入倒序与输出正常位序的结构



该结构特点：每一级结构都相同，仅支路增益不同
每一级输入可按照相同顺序存取

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

按时间抽取的FFT算法：把输入序列 $x(n)$ 在时域上进行分组

按频率抽取的FFT算法：把输出序列 $X(k)$ 在频域上进行分组

设序列点数 $N = 2L$ ， L 为整数。将 $X(k)$ 按 k 的奇偶分组前，

先将输入 $x(n)$ 按 n 的顺序分成前后两半：

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} \\ &= \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=N/2}^{N-1} x(n) W_N^{nk} \\ &= \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=0}^{\frac{N}{2}-1} x\left(n + \frac{N}{2}\right) W_N^{\left(n + \frac{N}{2}\right)k} \\ &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) W_N^{\frac{N}{2}k} \right] W_N^{nk}, \quad (\text{由于 } W_N^{\frac{N}{2}k} = (-1)^k) \\ &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + (-1)^k x\left(n + \frac{N}{2}\right) \right] W_N^{nk}, \quad k = 0, 1, \dots, N-1 \end{aligned}$$

若 N 点DFT按 k 的奇偶分组, 则可分为两个 $N/2$ 点的DFT

当 k 为偶数, 即 $k = 2r$ 时, $(-1)^k = 1$;

当 k 为奇数, 即 $k = 2r + 1$ 时 $(-1)^k = -1$ 。

这时 $X(k)$ 可分为两部分:

$$\begin{aligned} k \text{ 为偶数时: } X(2r) &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_N^{2nr} \\ &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_{\frac{N}{2}}^{nr}, \end{aligned}$$

$$\begin{aligned} k \text{ 为奇数时: } X(2r+1) &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^{n(2r+1)} \\ &= \sum_{n=0}^{\frac{N}{2}-1} \left\{ \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^n \right\} W_{\frac{N}{2}}^{nr}, \end{aligned}$$

其中: $r = 0, 1, \dots, \frac{N}{2} - 1$

可知上面两式均为 $N/2$ 点的DFT

由上述推导可知：

1) $x(n)$ 的 N 点DFT系数 $X(k)$ 的偶序列 $X(2r)$ ：

可由输入序列 $x(n)$ 前 $N/2$ 点与后 $N/2$ 点之和构成 $N/2$ 点序列，对该序列进行 $N/2$ 点DFT得到

2) $x(n)$ 的 N 点DFT $X(k)$ 的奇序列 $X(2r + 1)$ ：

序列 $x(n)$ 前 $N/2$ 点与后 $N/2$ 点之差并乘以系数 W_N^n 构成 $N/2$ 点序列，对该序列进行 $N/2$ 点DFT得到

定义两个 $N/2$ 点序列：

$$g[n] = x[n] + x[n + N/2]$$

$$h[n] = x[n] - x[n + N/2]$$

$$\text{其中： } n = 0, 1, \dots, N/2 - 1$$

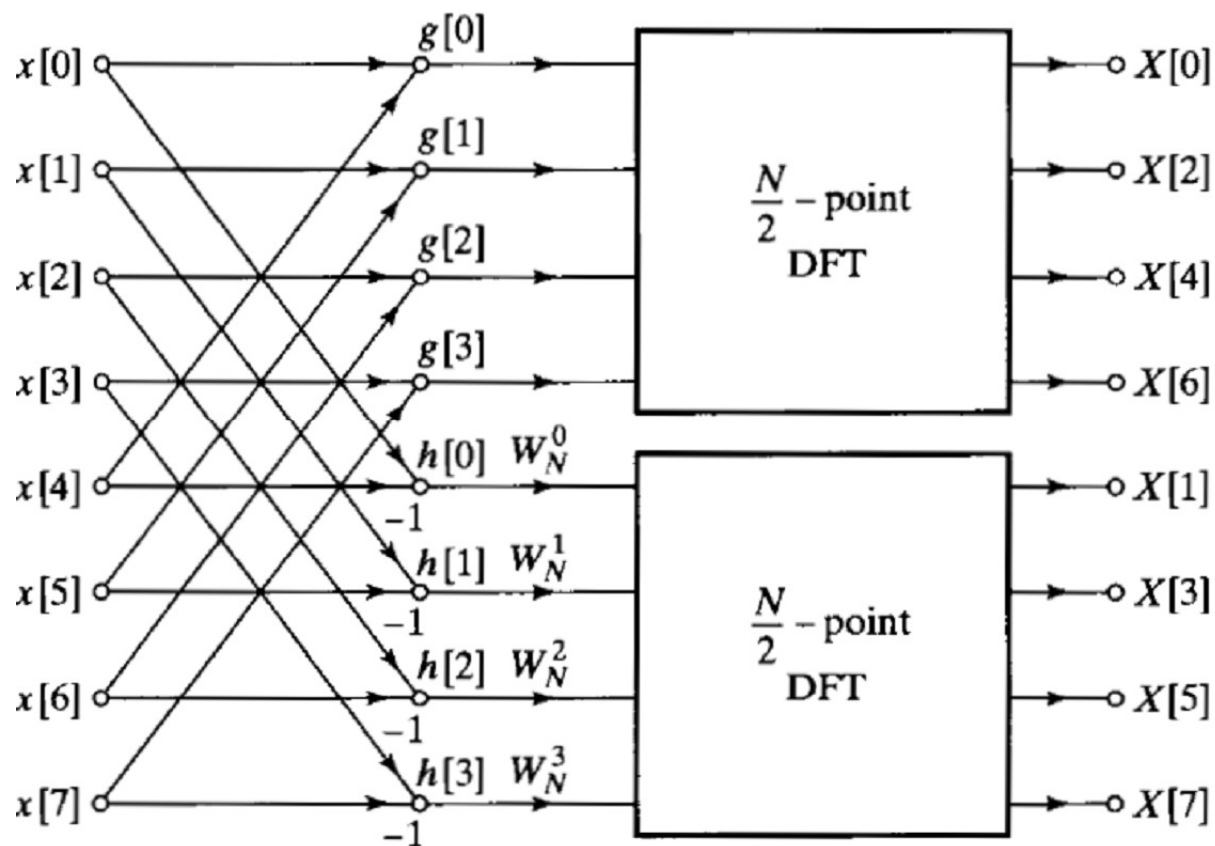
$$X(2r) = \sum_{n=0}^{\frac{N}{2}-1} g(n) W_{\frac{N}{2}}^{nr},$$

$$X(2r + 1) = \sum_{n=0}^{\frac{N}{2}-1} \left[h(n) W_N^n \right] W_{\frac{N}{2}}^{nr},$$

$$\text{其中： } r = 0, 1, \dots, \frac{N}{2} - 1$$

因此，有序列 $x(n)$ 的8点DFT的一次频率抽取的计算结构：

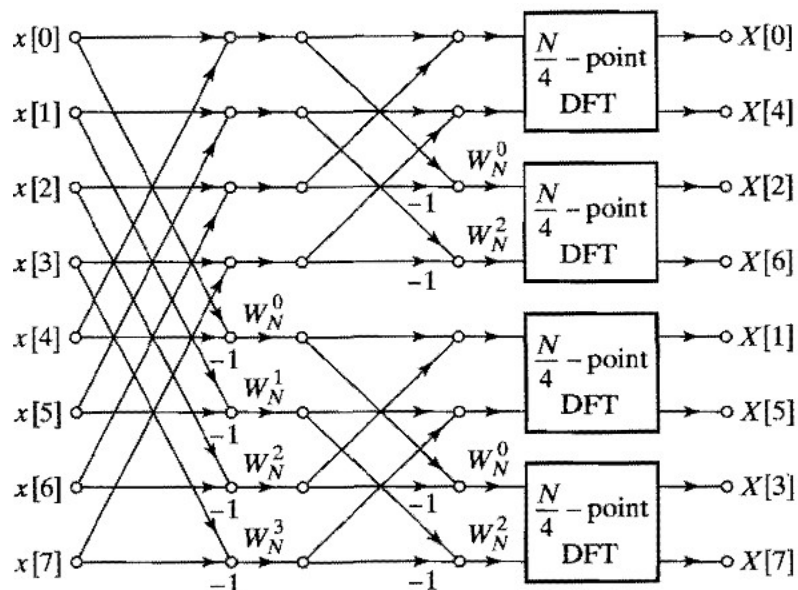
序列 $x(n)$ 的8点DFT的一次频率抽取的计算结构



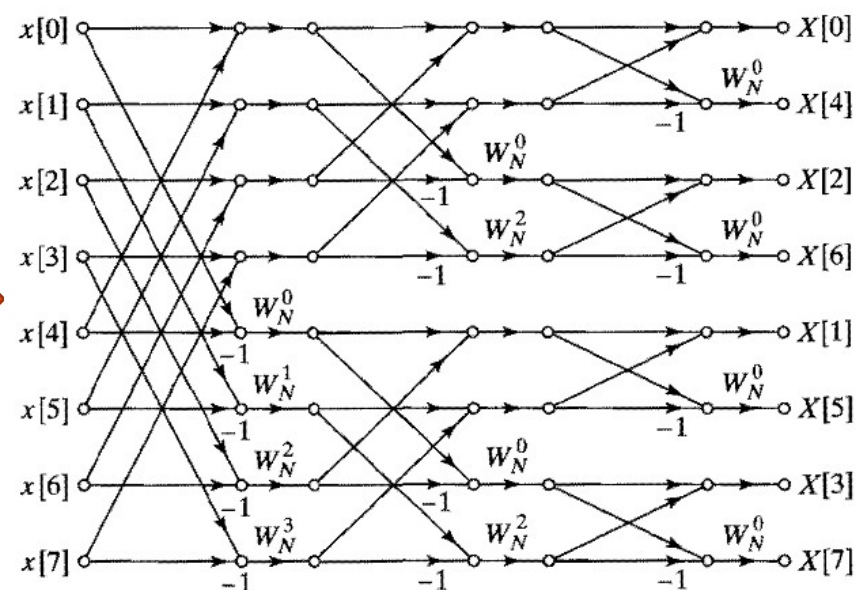
先计算 $g(n)$ 和 $h(n)$ ，再计算 $h(n)W_N^n$ ，最后分别计算 $N/2$ 点的DFT

如果类似时间抽取的方式，进一步执行频率分组，可得类似结构

针对 $N/2$ 点DFT继续进行频率分组



针对 $N/4$ 点DFT继续进行频率分组

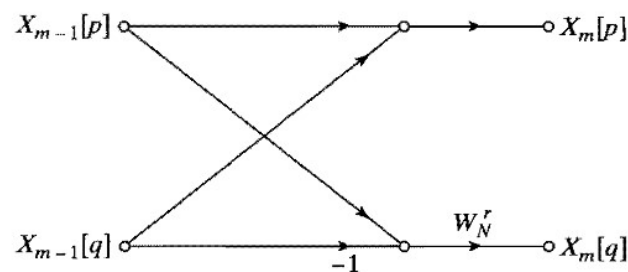


基本的蝶形运算：

$$X_m(p) = X_{m-1}(p) + X_{m-1}(q)$$

$$X_m(q) = [X_{m-1}(p) - X_{m-1}(q)]W_N^r$$

8点的基2频率抽取FFT计算结构



基2频率抽取算法与基2时间抽取算法比较:

- 1) 都由一系列蝶形运算组成
- 2) 蝶形运算的级数相同, 均为 v 级, 即 $N=2^v$
- 3) 每一级的蝶形运算数相同, 均为 $N/2$ 个
- 4) 蝶形运算的具体计算不同, 加减与乘系数次序相反
- 5) 整个算法的计算量相同, 均为
复乘次数: $(N/2)\log_2 N$
复加次数: $M\log_2 N$
- 6) 都具有同址计算特性
- 7) 倒序的位置不同, 但倒序方式相同,
- 8) 类似时间抽取算法, 还有很多其它结构形式

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel 算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

1、序列的排列顺序

时间抽取：输入倒序，输出顺序

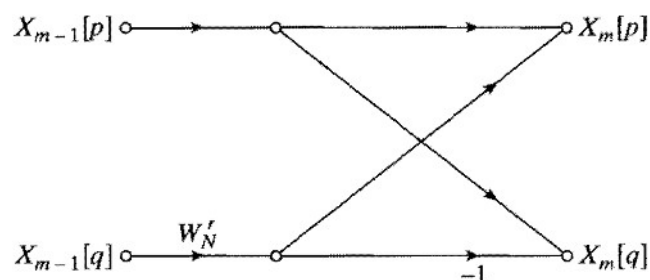
频率抽取：输入顺序，输出倒序

可进行同址计算，成对交换

根据实际的需求，选择时间抽取或频率抽取算法

2、蝶距

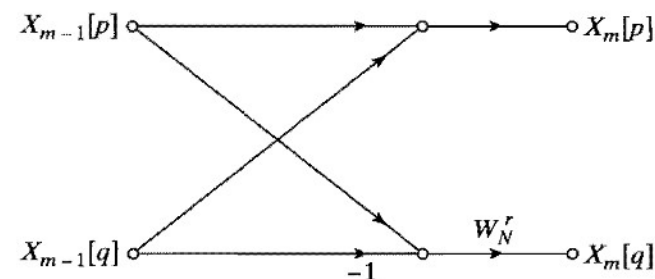
时间抽取



第m级蝶形运算两点间的距离：

$$q - p = 2^{m-1}$$

频率抽取



第m级蝶形运算两点间的距离：

$$q - p = N \cdot 2^{-m} = 2^{v-m}$$

3、系数 W_N^r 的确定

时间抽取FFT的蝶形运算

$$X_m(p) = X_{m-1}(p) + W_N^r X_{m-1}(q)$$

$$X_m(q) = X_{m-1}(p) - W_N^r X_{m-1}(q)$$

频率抽取FFT的蝶形运算

$$X_m(p) = X_{m-1}(p) + X_{m-1}(q)$$

$$X_m(q) = [X_{m-1}(p) - X_{m-1}(q)]W_N^r$$

W_N^r 的计算:

- 1) 建立系数表;
- 2) 用正、余弦函数;
- 3) 递推方法: 因任何一级的系数可表示为复数幂次方的形式

$$\text{即: } W_N^{ql} = W_N^q W_N^{q(l-1)}$$

$$\text{例: } W_8^1 \longrightarrow W_8^2 = W_8^1 W_8^1 \longrightarrow W_8^3 = W_8^2 W_8^1$$

4、IDFT的快速算法

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{nk}$$

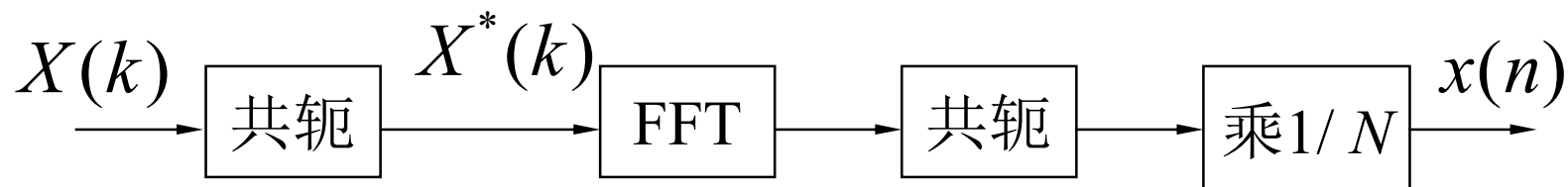
$$x(n) = IDFT[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk}$$

1) 方法一:

用 W_N^{-kn} 代替 W_N^{kn} ，最后再乘以常数 $1/N$ 就可以得到 IDFT 的快速算法——IFFT。

2) 方法二:

$$x^*(n) = \left[\frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \right]^* = \frac{1}{N} \sum_{k=0}^{N-1} X^*(k) W_N^{nk} = \frac{1}{N} \left\{ DFT[X^*(k)] \right\}^*$$



5、一般 N 值的FFT算法

N 为复合数时，即： $N = RQ$ ，可采用混合基算法

把 N 点DFT表示为 R 个 Q 点DFT的和，或 Q 个 R 点DFT的和

6、基4-FFT算法

速度比基2-FFT算法快，基4运算比基2运算节省 25% 的复乘次数，但复加次数增加 50%；

其它算法：比如混合基FFT算法、分裂基FFT算法（课后习题）

例子：基4 FFT算法

1) Radix 4算法

(1)算法原理：设 $N=4^3=64$ ，用DIT—FFT做第一次分解，

$$\text{令 } N=16 \times 4, \quad 0 \leq m_0 \leq 15, \quad 0 \leq n_2 \leq 3$$

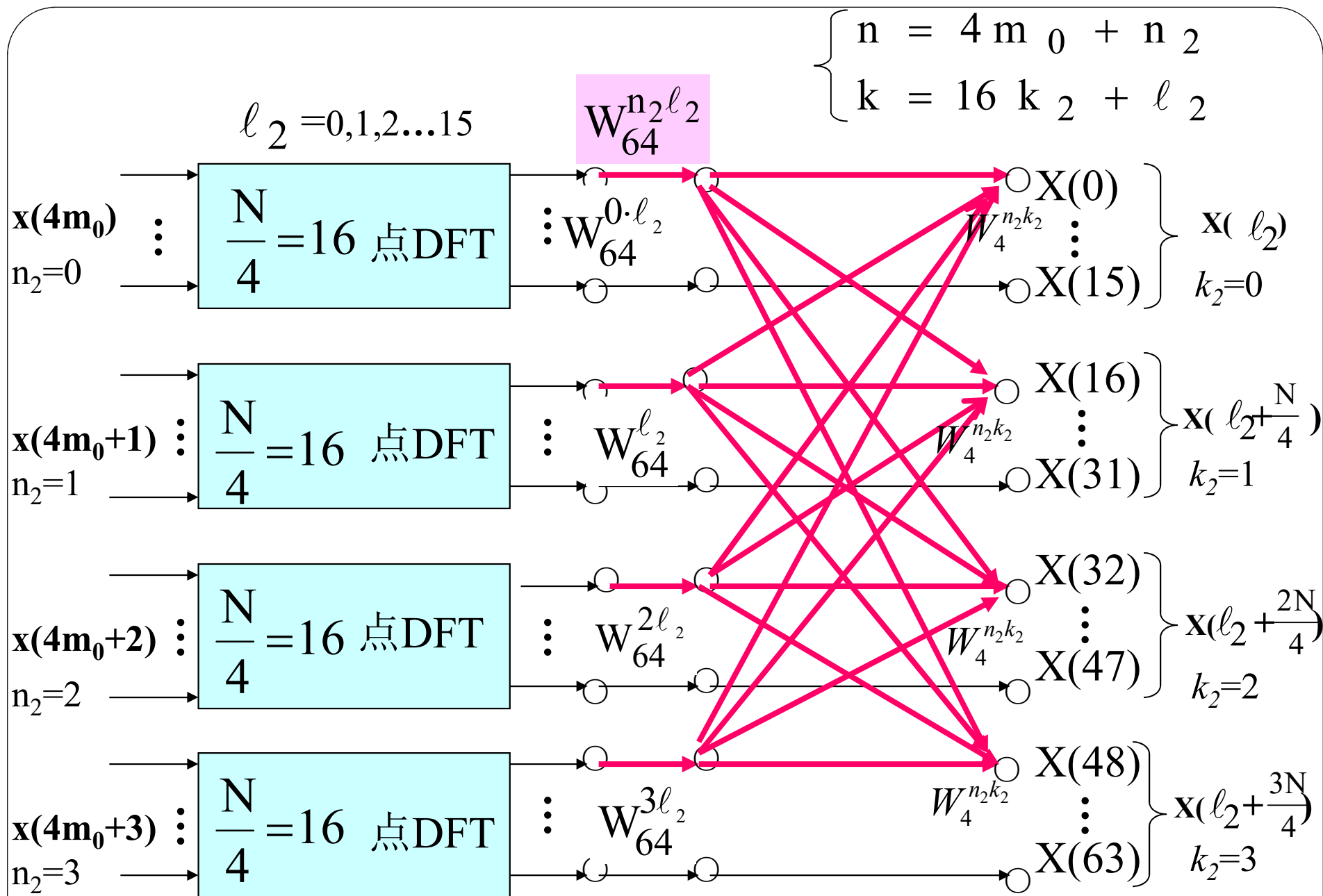
$$\begin{cases} n = 4m_0 + n_2 \text{ (时域分组)} & 0 \leq \ell_2 \leq 15, \quad 0 \leq k_2 \leq 3 \\ k = 16k_2 + \ell_2 \text{ (频域分组)} \end{cases}$$

$$X(k) = \sum_{n_2=0}^3 \sum_{m_0=0}^{15} x(4m_0 + n_2) W_{64}^{(4m_0 + n_2)(16k_2 + \ell_2)}$$

$$= \sum_{n_2=0}^3 \sum_{m_0=0}^{15} x(4m_0 + n_2) W_{16}^{m_0 \ell_2} W_{64}^{n_2 \ell_2} W_4^{n_2 k_2}$$

16点DFT

DFT后组合运算所
需要乘的2个系数



2) 基4算法特点:

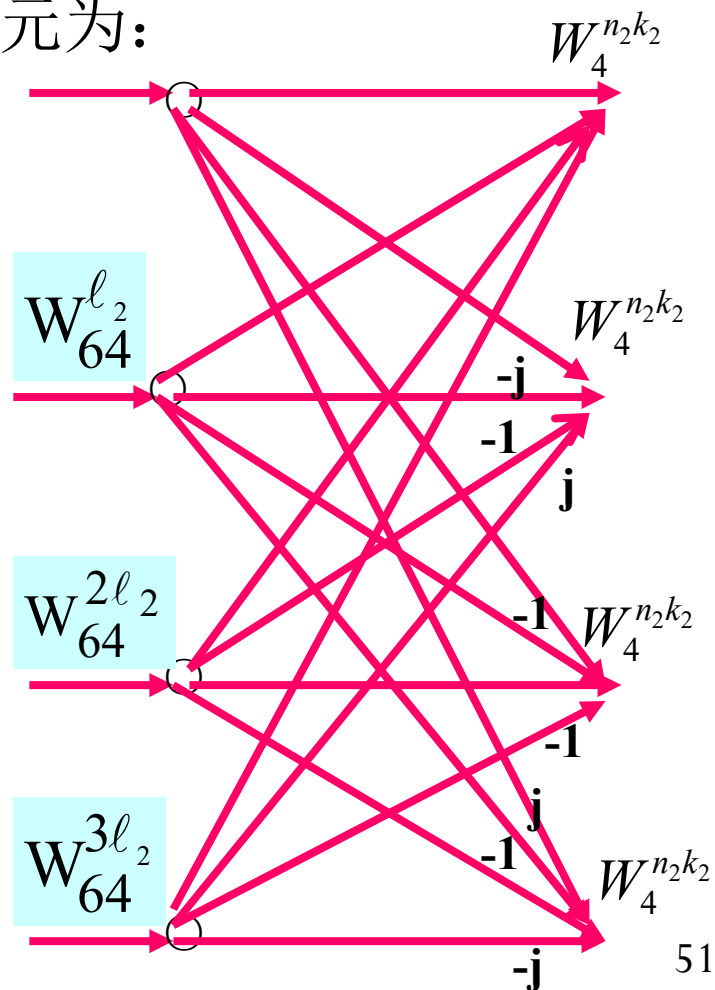
① 若点数 $N=4^m$, 则基4分解的级数是 m 级, 且每级间都是同位运算;

② 乘法次数: 基4蝶形运算单元为:

- 每一基4蝶形运算

需3次复乘, 12次复加

- 每级共有 $\frac{N}{4}$ 个基4蝶形;
- 共有 m 级。



∴ 基4运算所需的复乘次数和复加次数分别是：

$$\begin{aligned} 3 \times \frac{N}{4} \times (m-1) &\approx \frac{3}{4} Nm = \frac{3}{4} N \log_4 N \\ &= \frac{3}{4} N \left(\frac{1}{2} \log_2 N \right) = \frac{3}{4} \left(\frac{N}{2} \log_2 N \right) = \frac{3}{4} \times (\text{基2复乘次数}) \end{aligned}$$

$$12 \times \frac{N}{4} \times \log_4 N = 3N \log_4 N = \frac{3}{2} N (\log_2 N)$$

结论：基4运算比基2运算节省 25% 的复乘次数，但复加次数增加 50%；

- 作业: P9.6, P9.21, P9.39, P9.45

概要

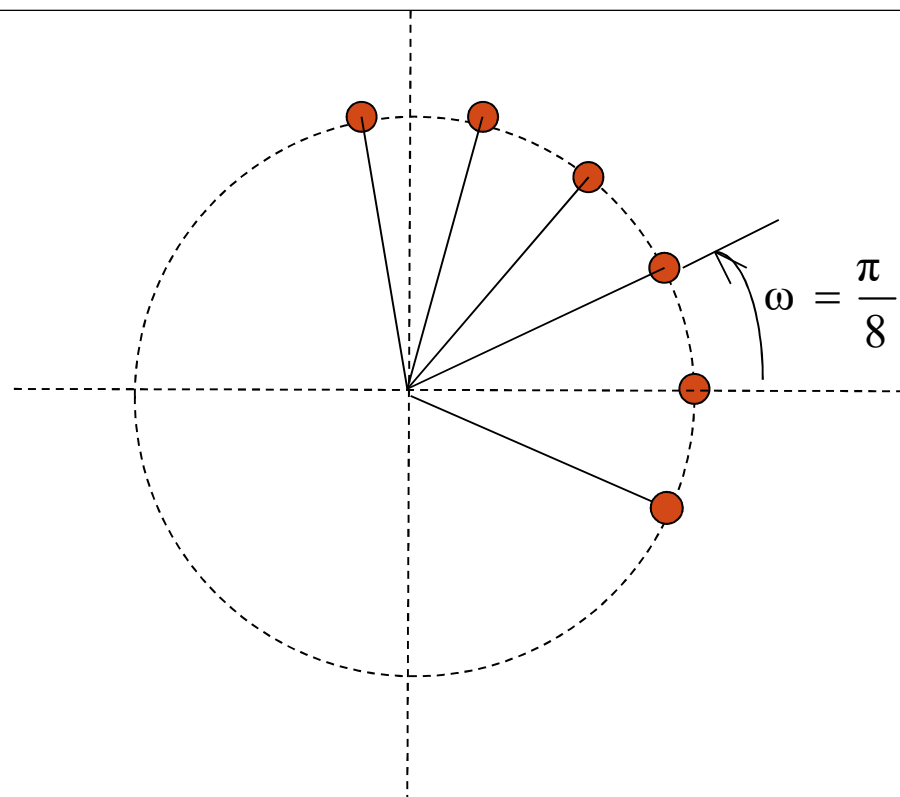
- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel 算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

线性调频Z变换算法（CTA或CZT）

1. FFT的局限性：

FFT：输入 N 点，输出 N 点，均匀分布于单位圆，频域分辨率为 $2\pi/N$ ；

- 1) 当输入点数极少时，若希望某频带内频域采样点较多(分辨率更高)，则需要补零，增加了计算量；
- 2) 对于窄带信号，希望通带内频率部分点密，带外疏，或根本不用计算，用FFT计算需计算全部点，再取一部分；
- 3) 难以准确得到系统的“自然频率”位置；
- 4) 希望准确计算 N 点DFT，但 N 是大的素数时。



$$z_k = e^{j2\pi k/N}$$

做DFT时， z 变换在单位圆上的等分的 N 个点上取值。

例：要求计算 $\omega = 0 \rightarrow \frac{\pi}{8}$

内 $L=64$ 点的DFT，用FFT完成，则必须在整个单位圆上采样

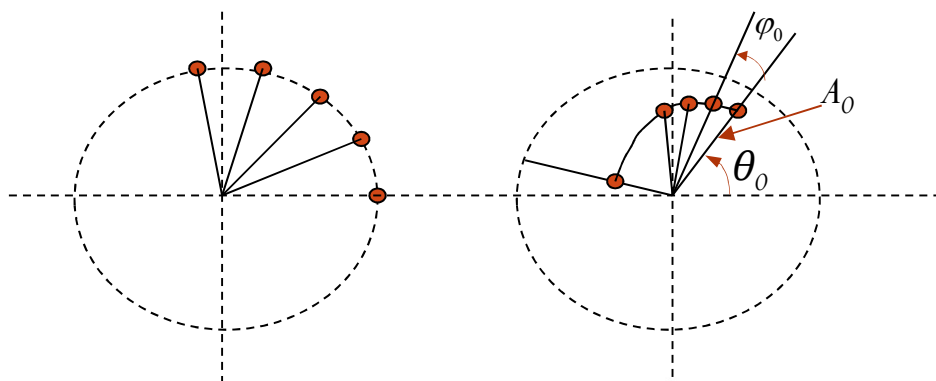
$$64 \times 16 = 1024 = 2^{10} \text{ 点,}$$

原始序列补零到1024点，

做 2^{10} 点的FFT，取 $0 \sim 63$ 点，即为要求的值。

2、CZT算法原理

它是计算螺旋线周线上的z变换采样，



$$z_k = e^{j2\pi k/N}$$

做DFT时，
z变换在单位圆上的等分的N个点上取值。

$$z_k = A_0 e^{j\theta_0} W_0^{-k} e^{jk\varphi_0}$$

CZT时，
离散路径可在单位圆内、外，或上面。

$$X(z)|_{z=z_k} = \sum_{n=0}^{N-1} x(n) z_k^{-n}$$

其中

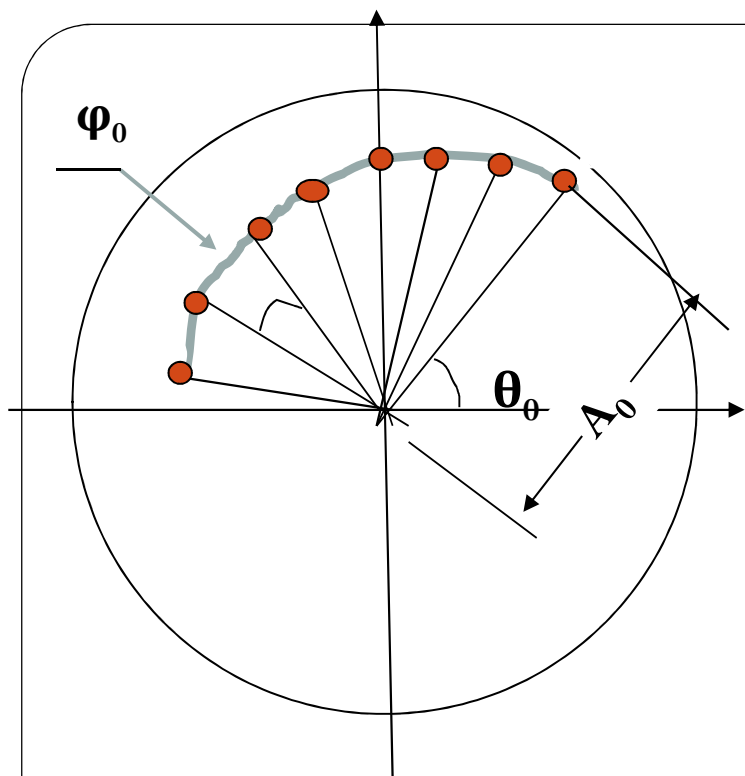
$$z_k = A W^{-k}, \quad 0 \leq k \leq (M-1)$$

$$z_k = (A_0 e^{j\theta_0}) (W_0 e^{-j\varphi_0})^{-k}$$

$W_0 > 1$ ，螺旋线向中心弯曲；

$W_0 < 1$ ，螺旋线远离中心弯曲；

$W_0 = 1, A_0 = 1$ ，是单位圆上一部分；



$$z_k = A_0 e^{j\theta_0} W_0^{-k} e^{jk\varphi_0}$$

若: $\theta_0 = 0$, $W_0 = A_0 = 1$,

$$\varphi_0 = \frac{2\pi}{N}, \quad M = N$$

即是DFT。

CZT在Z平面上的变换
路径是一条螺旋线

A_0, θ_0 决定CZT的起点;

W_0 决定变换路径如何倾斜

φ_0 决定变换的步长。

信号的点数 N 和变换路
径的点数 M 可以不相等。

CZT的特点

- CZT可计算单位圆上任一段曲线上的 z 变换，可任意给定起止频率；
- 作变换时输入的点数 N 和输出点数 M 可以不相等；
- 可达到对任意频率区域“细化”的目的。
- CZT可认为是“一般性”的DFT

3.利用CZT算法计算:

$$X(k) = X(z_k) = \sum_{n=0}^{N-1} x(n) z_k^{-n} = \sum_{n=0}^{N-1} x(n) A^{-n} W^{nk} \quad (0 \leq k \leq M-1)$$

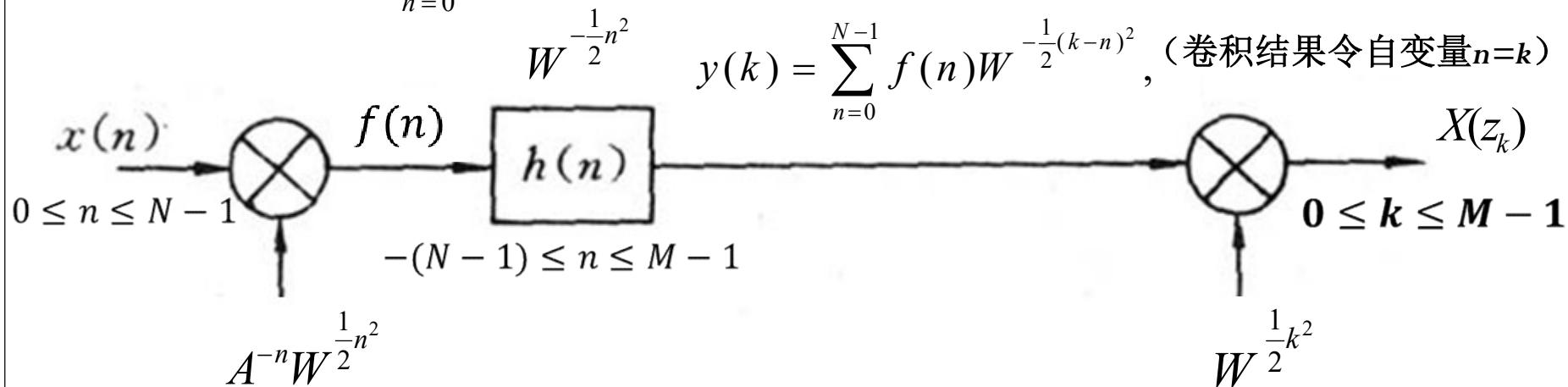
利用 $nk = \frac{1}{2}[n^2 + k^2 - (k-n)^2]$

$$\begin{aligned} X(z_k) &= \sum_{n=0}^{N-1} x(n) A^{-n} W^{\frac{1}{2}n^2} W^{-\frac{1}{2}(k-n)^2} W^{\frac{1}{2}k^2} \\ &= W^{\frac{1}{2}k^2} \sum_{n=0}^{N-1} \underbrace{(x(n) A^{-n} W^{\frac{1}{2}n^2})}_{f(n)} \underbrace{W^{-\frac{1}{2}(k-n)^2}}_{h(k-n)} \end{aligned}$$

序列 $h(n)$ 自变量取值范围:
 $-(N-1) \leq n \leq M-1$

$$X(z_k) = W^{\frac{1}{2}k^2} \sum_{n=0}^{N-1} f(n) h(k-n), \quad (0 \leq k \leq M-1)$$

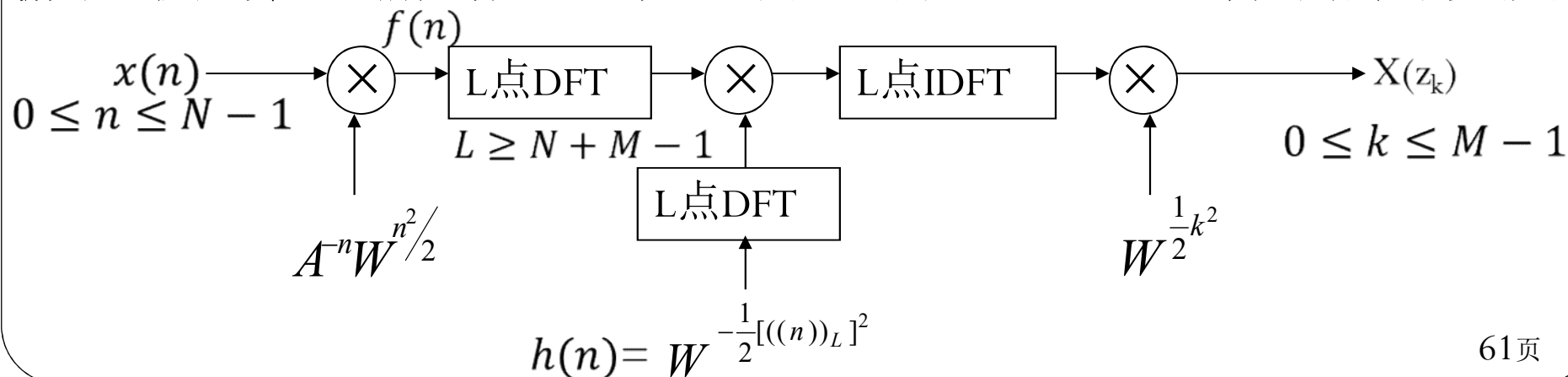
$$X(z_k) = W^{\frac{1}{2}k^2} \sum_{n=0}^{N-1} f(n)h(k-n) \quad \text{CZT直接实现框图}$$



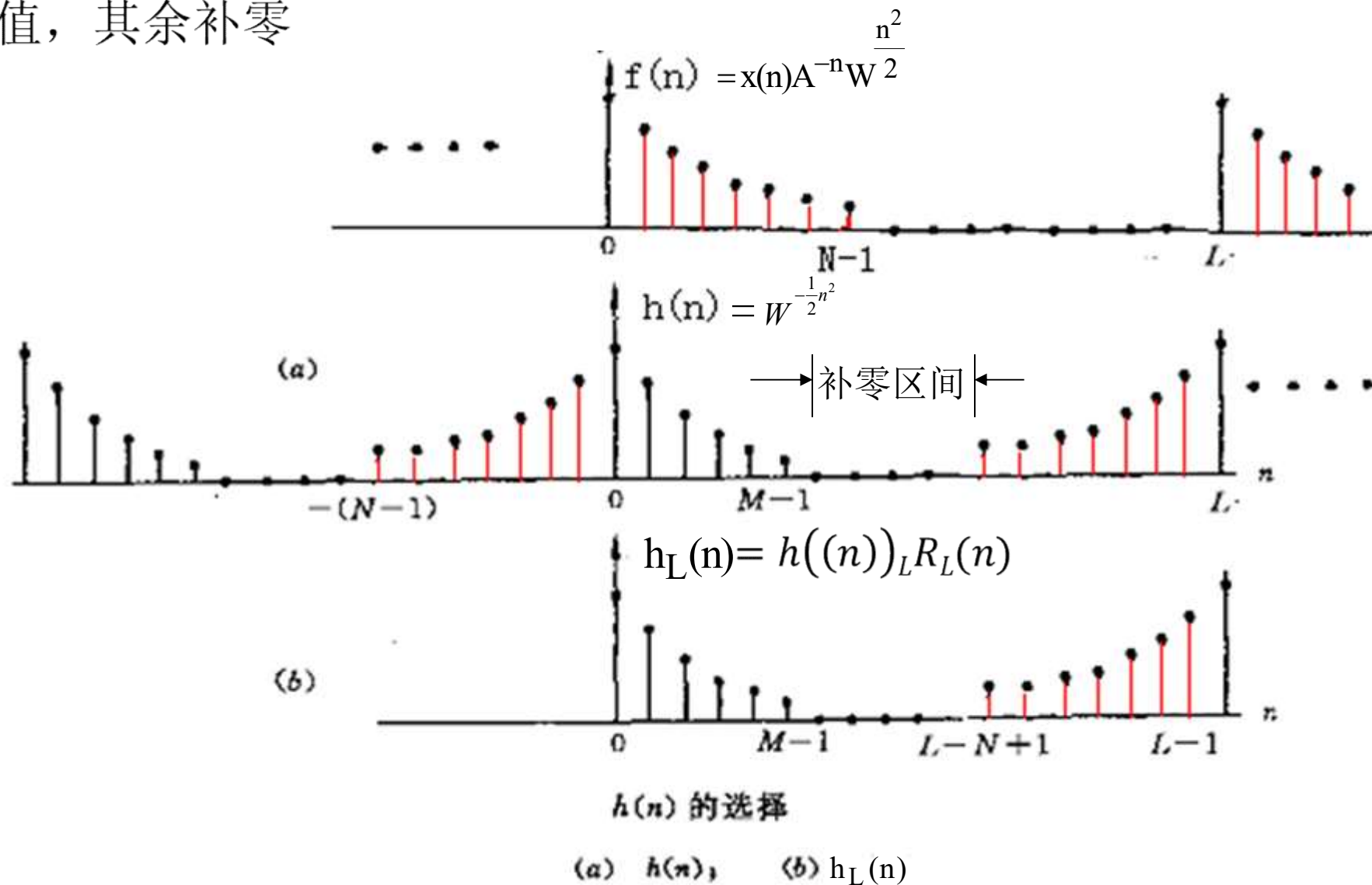
用DFT（FFT）实现CZT算法框图

$$X(z_k) = W^{\frac{1}{2}k^2} \cdot f(k) \otimes h(k), \text{循环卷积长度为 } L \geq N + M - 1$$

原因：序列 $f(k)$ 与 $h(k)$ 的线性卷积结果在区间 $-(N-1) \leq k \leq N+M-2$ 上，而循环卷积只会重叠前后各 $N-1$ 个点，因此区间 $0 \leq k \leq M-1$ 内的结果不受影响



4、用FFT计算时， $h(n)$ 的取值范围： $-(N-1) \leq n \leq M-1$ 上有值，其余补零



5、CZT计算步骤及乘法次数统计：

(1) 选择可用FFT计算的点数，取 $L = 2^m$

(2) 计算

$$f(n) = \begin{cases} x(n)[A^{-n}W^{n^2/2}] & 0 \leq n \leq N-1 \\ 0 & N \leq n \leq L-1 \end{cases}$$

N次乘法

(3) 计算 $DFT[f(n)] = F(k)$

$L/2 \log_2 L$

(4) 计算

$$h(n) = \begin{cases} W^{-n^2/2} & 0 \leq n \leq M-1 \\ 0 & M \leq n \leq L-N \\ W^{-(L-n)^2/2} & L-N+1 \leq n \leq L-1 \end{cases}$$

(5) 计算 $DFT[h(n)] = H(k) \quad 0 \leq k \leq L-1$

预先算得

(6) 计算 $H(k)F(k)$

L次乘法

(7) 计算 $IDFT[H(k)F(k)] = y(k)$

$L/2 \log_2 L$

(8) 取 $y(k)$ L点中的M点： $X(z_k) = y(k)W^{\frac{1}{2}k^2}$

M次乘法

乘法次数统计：

$$\begin{aligned} Mc &= N + \frac{L}{2} \log_2 L + L + \frac{L}{2} \log_2 L + M \\ &= (N + M) + (N + M - 1) + (N + M - 1) \log_2 (N + M - 1) \\ &\approx (N + M - 1) [2 + \log_2 (N + M - 1)] \end{aligned}$$

例： 1. $N = 481$, $M = 32$ $L = 481 + 32 - 1 = 512 = 2^9$

直接计算需复乘次数： $NM = 481 \times 32 = 15392$

用CZT算法： $512[2 + \log_2(2^9)] = 512 \times 11 = 5632$

2. $N = 25$, $M = 8$, $L = 25 + 8 - 1 = 32 = 2^5$

直接计算需复乘次数： $NM = 25 \times 8 = 200$

用CZT算法： $32[2 + \log_2(2^5)] = 32 \times 7 = 224$

• 要求的点数大时用**CZT**算法，相对小时用直接计算。

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel 算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响

FFT算法的有限字长效应

- 从运算的角度分析，FFT与数字滤波器一样都是数字系统，因此二者的有限字长效应分析方法基本上是相同的
- 不同的是FFT中的运算是一个复数运算，因此误差源是一个复数，同时FFT运算有多个输入端及多个输出端，各误差源对不同的输出端的影响是不同的。
- 以时间抽取DIT(Decimation In Time)为例进行有限字长效应分析，并且针对的是舍入情况，其他FFT算法及截尾运算结果是相似的。

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响
 - 1、FFT量化误差分析
 - 2、FFT溢出问题与输出信噪比
 - 3、FFT浮点运算量化效应

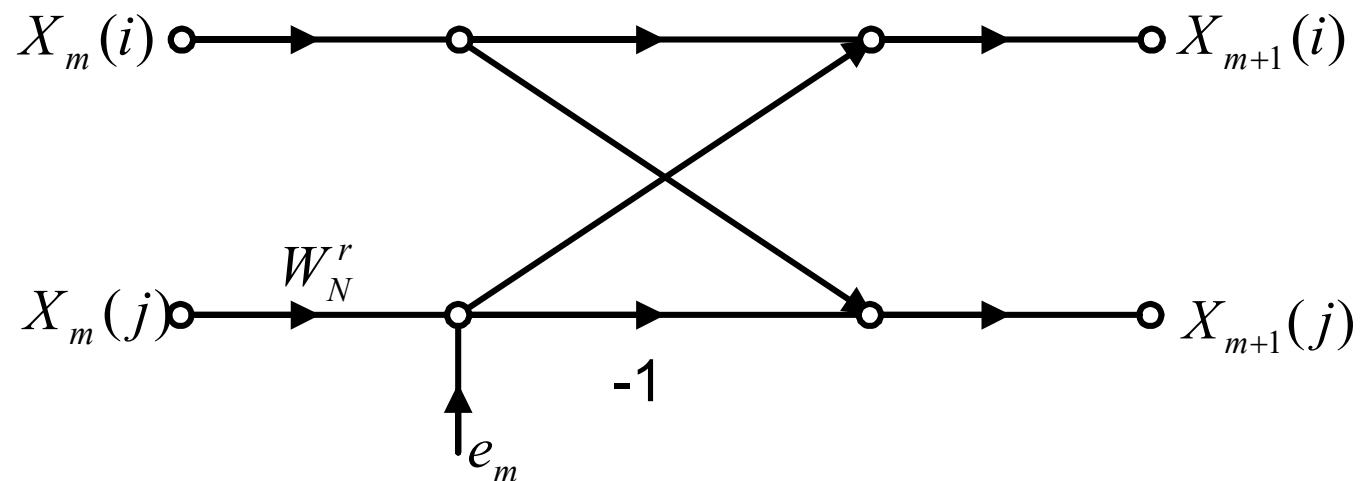
蝶形运算的统计模型

$$X_{m+1}(i) = X_m(i) + W_N^r X_m(j)$$

$$X_{m+1}(j) = X_m(i) - W_N^r X_m(j)$$

定点制运算中只有乘法引入量化误差，加法运算不引入误差

蝶形运算中只在乘系数 W_N^r 时引入一个误差源 e_m



$W_N^r X_m(j)$ 是复数相乘，所产生误差 e_m 就是一个复数

一个复数相乘要由四个实数相乘来构成、每个实数相乘都将引入一个相应的误差。因而共有四个误差 e_1, e_2, e_3, e_4

假设是舍入量化，则有：

$$\begin{aligned} Q_r[X_m(j)W_N^r] &= X_m(j)W_N^r + e_m \\ &= \{\text{Re}[X_m(j)]\text{Re}[W_N^r] + e_1 - \text{Im}[X_m(j)]\text{Im}[W_N^r] + e_2\} \\ &\quad + j\{\text{Re}[X_m(j)]\text{Im}[W_N^r] + e_3 + \text{Im}[X_m(j)]\text{Re}[W_N^r] + e_4\} \\ e_m &= e_1 + e_2 + j(e_3 + e_4) \end{aligned}$$

假设这四个误差 ($e_1 \sim e_4$) 为互不相关的零均值加性白噪声:

定义复数 e_m 的方差 σ_B^2 为:

$$\begin{aligned}\sigma_B^2 &= E[(e_m - u_m)(e_m - u_m)^*] \\ &= E[|e_m|^2] = E[|e_1|^2] + E[|e_2|^2] + E[|e_3|^2] + E[|e_4|^2] \\ &= 4\sigma_e^2 = \frac{q^2}{3}, \text{ 其中 } q = 2^{-B} X_m = \Delta, \text{ 即量化步长}\end{aligned}$$

分析乘以 W_N^r 后的方差影响:

$$E[|e_m W_N^r|^2] = |W_N^r|^2 \cdot E[|e_m|^2] = \sigma_B^2$$

e_m 通过所有后级蝶形时, 其方差保持不变

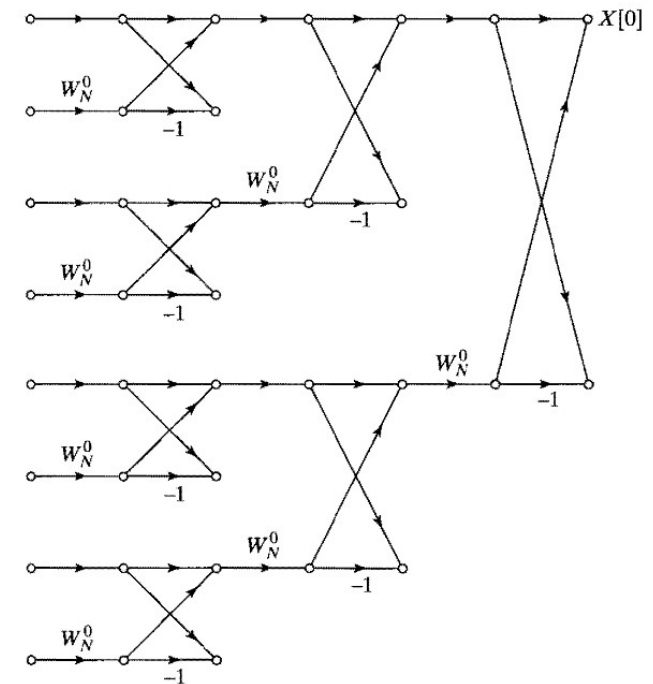
每一个输出端都与 $N-1$ 个蝶形相连，
即有 $N-1$ 个量化噪声源对每个输出端有贡献

在输出端，即离散傅里叶变换 $X(k)$

上叠加的输出噪声的方差为：

$$\sigma_F^2 = (N-1)\sigma_B^2 \approx N\sigma_B^2 = \frac{Nq^2}{3}$$

和FIR滤波器的直接型实现一样，输出噪声的总方差正比于 N



概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响
 - 1、FFT量化误差分析
 - 2、FFT溢出问题与输出信噪比
 - 3、FFT浮点运算量化效应

防止溢出和FFT输出的信噪比

分析蝶形运算： $X_{m+1}(i) = X_m(i) + W_N^r X_m(j)$

$$|X_{m+1}(i)| \leq |X_m(i)| + |W_N^r| |X_m(j)| = |X_m(i)| + |X_m(j)|$$

$$\text{可得： } |X_{m+1}(i)| \leq |X_m(i)| + |X_m(j)|$$

$$\text{所以： } \max\{|X_{m+1}(i)|, |X_{m+1}(j)|\} \leq 2 \max\{|X_m(i)|, |X_m(j)|\}$$

$$\text{还有： } \max\{|X_{m+1}(i)|, |X_{m+1}(j)|\} \geq \max\{|X_m(i)|, |X_m(j)|\}$$

蝶形运算的输出最大值不超过输入端最大值的两倍。

从前一级到后一级，最大模值是逐级非减的，只要最后一级不出现溢出，则前一级计算一定不会溢出。

三种防止溢出的办法

1、输入端一次衰减法

由DFT公式可知：

$$|X(k)| = \left| \sum_{n=0}^{N-1} x(n) W_N^{nk} \right| \leq \sum_{n=0}^{N-1} |x(n)| \leq 1$$

为使 $X(k)$ 不溢出， 所以输入 $x(n)$ 应满足条件：

$$|x(n)| \leq \frac{1}{N}$$

输入端衰减因子 s ， 应满足：

$$s \leq \frac{1}{Nx_{\max}}$$

若 $|x(n)|$ 在 $(-1/N, 1/N)$ 内等概率分布, 则 $x(n)$ 的方差为:

$$\sigma_x^2 = E[|x(n)|^2] = \frac{1}{3N^2}$$

由于 $X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}$, 输出信号的方差为:

$$E[|X(k)|^2] = E\left\{\sum_{n=0}^{N-1}\sum_{m=0}^{N-1} x(n)W_N^{kn} x^*(m)W_N^{-km}\right\} \quad \text{假设 } x(n) \text{ 是白噪声}$$

$$= \sum_{n=0}^{N-1} E[|x(n)|^2] = N\sigma_x^2 = \frac{1}{3N}$$

输出信噪比为:

$$SNR = \frac{E[|X(k)|^2]}{\sigma_F^2} = \frac{1}{N^2 q^2} = N^{-2} 2^{2B} \quad (\because \sigma_F^2 \approx \frac{Nq^2}{3}, q = 2^{-B})$$

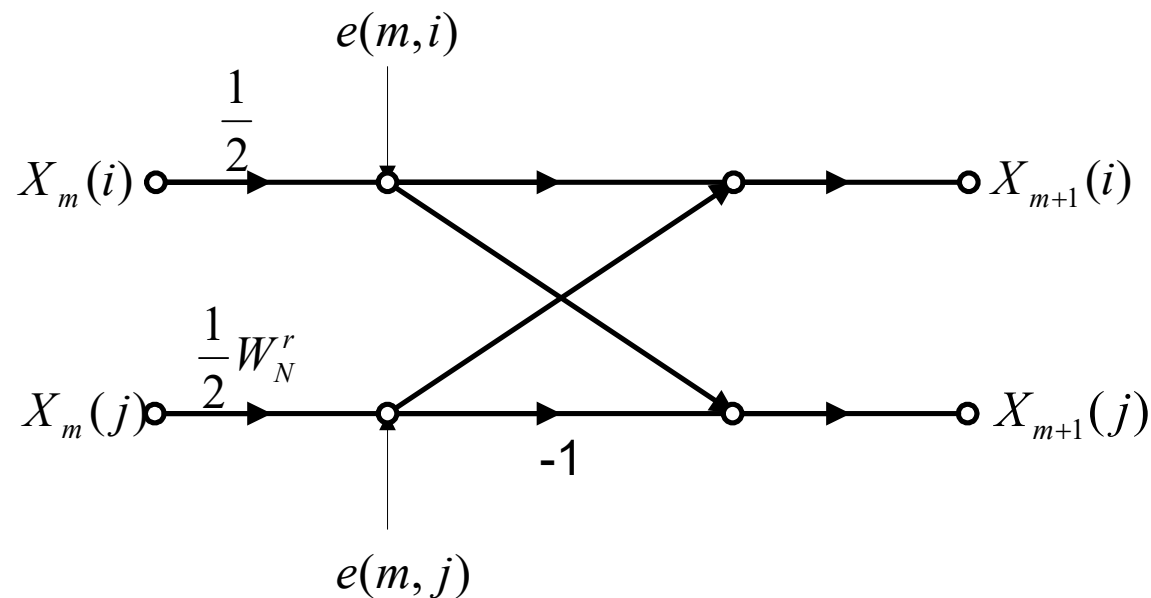
输出信噪比与 N^2 成反比, N 增加一倍, SNR 下降4倍

若保持运算精度不变, 每增加一级运算 ($2N$), 字长也需相应增加一位。

这种防止溢出的办法, 使得输入幅度被限制得过小, 造成输出信号 / 噪声比值过小

2、逐级衰减法

由于： $\max\{|X_{m+1}(i)|, |X_{m+1}(j)|\} \leq 2 \max\{|X_m(i)|, |X_m(j)|\}$
 对每个蝶形结构的两个输入支路都乘上个比例因子



每一输入信号均乘以1/2，多引入一个噪声源，有二个误差源 e_{m1} 、 e_{m2} ，每个误差源的方差均近似为：

$$\sigma_B^2 = \frac{q^2}{3} = \frac{2^{-2B}}{3}$$

则每个蝶形运算总的误差方差为：

$$\sigma_{B'}^2 = E[|e_{m1}|^2] + E[|e_{m2}|^2] = 2\sigma_B^2$$

由于系数1/2存在，各误差源到输出端的传输系数不再是 ± 1 或 W_N^r 的连乘积，

而是 $\frac{1}{2}W_N^r$ 或 $-\frac{1}{2}W_N^r$ 的连乘积，连乘次数与蝶形位置有关。

因此第 m 级蝶形的一个误差源在输出端的误差方差 σ_Q^2 为：

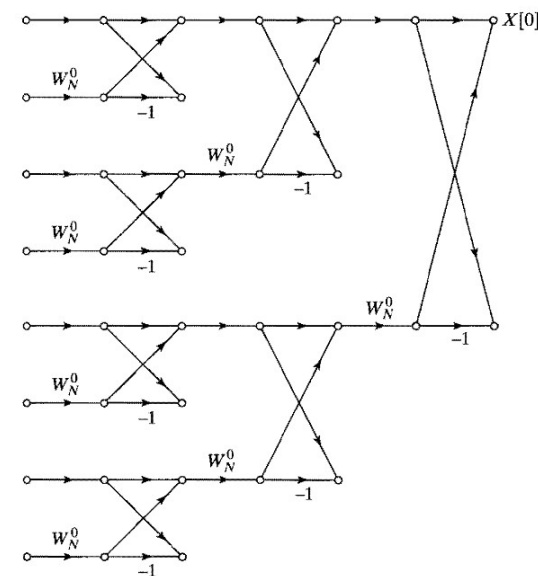
$$\sigma_Q^2 = \sigma_B^2 \cdot 2^{-2(v-m-1)}, \text{ 其中 } N = 2^v, m = 0, 1, \dots, v-1$$

总的输出噪声方差为：

$$\begin{aligned}
 \sigma_F^2 &= \sum_{m=0}^{v-1} 2^{v-m} \cdot \sigma_Q^2 = \sum_{m=0}^{v-1} 2^{v-m} \sigma_B^2 2^{-2(v-m-1)} \\
 &= \sum_{m=0}^{v-1} 2^{-(v-m-2)} \sigma_B^2 \\
 &= 4\left(1 - \frac{1}{N}\right) \sigma_B^2, N \text{ 值较大时有:} \\
 &\approx 4 \sigma_B^2 = \frac{4}{3} q^2
 \end{aligned}$$

输出信噪比为：（假设 $|x(n)|$ 在 $(-1,1)$ 内等概率分布）

$$SNR = \frac{E[|X(k)|^2]}{\sigma_F^2} = \frac{1/(3N)}{4q^2/3} = \frac{1}{4Nq^2}$$



与输入端一次衰减法相比，信噪比有了很大的提高

为保证运算精度不变， N 增加4倍，字长只需增加一位

3、块浮点运算

将原始数据用成组浮点制表示为

$$\text{Re}[x(i)] = 2^p \text{Re}[x'(i)] \quad \text{Im}[x(i)] = 2^p \text{Im}[x'(i)]$$

保证 $\text{Re}[x'(i)] < 1$, $\text{Im}[x'(i)] < 1$

p 为某个整数, 称为共阶数, $i = 0, 1, \dots, N - 1$

对 $[x'(i)]$ 作定点FFT, 当某个蝶形结出现溢出, 则将整个这一级的序列(运算过的, 未运算的)全部右移一位(即乘 $1/2$ 因子), 并在阶码 p 上加1, 然后运算从发生溢出的蝶形继续下去。

当以后某级蝶形运算又出现溢出时, 再对该级的输入乘上 $1/2$ 衰减因子, 并在阶码 p 上加1继续运算, 直到 L 级计算完为止

如果在整个变换过程中，一次溢出也没有发生，输出信号方差最大，当输入为(-1,1)分布的白噪声型信号时，其值为：

$$\sigma_X^2 = N\sigma_x^2 = \frac{N}{3} \quad (|x(n)| < 1 \text{ 时})$$

输出信噪比为：

$$\text{SNR} = \frac{\sigma_X^2}{\sigma_F^2} = \frac{N/3}{N\sigma_B^2} = \frac{1}{q^2}, \sigma_B^2 = \frac{q^2}{3}$$

最坏的情况是各级都产生了衰减，这相当于第二种方法。所以成组浮点法快速傅立叶变换的信噪比变化范围为：

$$\frac{1}{4Nq^2} \leq \text{SNR} \leq \frac{1}{q^2}$$

概要

- 9.1、离散傅里叶变换的直接计算
- 9.2、Goertzel算法
- 9.3、按时间抽取的FFT算法
- 9.4、按频率抽取的FFT算法
- 9.5、实现问题的考虑
- 9.6、用卷积实现DFT
- 9.7、有限寄存器长度的影响
 - 1、FFT量化误差分析
 - 2、FFT溢出问题与输出信噪比
 - 3、FFT浮点运算量化效应

3、浮点FFT算法中的量化效应

- 采用的分析方法基本相同
 - ✓ 对输入信号和舍入噪声做同样的假设
 - ✓ 对任一输出节点，只需考虑与其相连的那些蝶形单元
- 与定点FFT算法误差分析不同之处有：
 - ✓ 浮点加法和浮点乘法之后都需做尾数处理，都会引入噪声源，因此模型设计与定点FFT系统有所不同
 - ✓ 模型中的绝对误差需换算成相对误差，再进行分析
 - ✓ 浮点制FFT系统任一输出节点处的输出信噪比为：

$$\frac{E\{|X(k)|^2\}}{E\{|e_f(k)|^2\}} = \frac{1}{2\sigma_e^2 \log_2 N}$$

浮点计算:

- 如果采用浮点计算, 信噪比正比于 $1/\log_2 N$, 而定点计算则正比于 $1/N$
- 对于精度要求高的地方就需要用浮点来计算

- 作业: P9.26, P9.51